CHAPTER 1
INTRODUCTION

The rapid digitization has led to the widespread adoption of biometric attendance systems. These systems improve efficiency, accuracy, and transparency by eliminating manual roll calls. However, they also handle highly sensitive data such as biometric templates, identities, and attendance records, which require strong security protection. Most existing systems rely heavily on centralized cloud architectures and legacy biometric devices that were not designed with modern cybersecurity principles. This creates multiple attack surfaces, including insecure data transmission, a lack of authentication, and dependency on constant internet connectivity. This project addresses these challenges by analyzing existing vulnerabilities and proposing a secure and scalable solution.

Key challenges in existing biometric attendance systems include:

- Transmission of biometric data over unencrypted or weakly secured networks.
- Vulnerability to replay attacks, data interception, and unauthorized access.
- Heavy dependence on continuous internet connectivity.
- Risk of data loss and system downtime during network failures.
- Irreversible nature of biometric data once compromised.

These issues highlight the importance of securing biometric attendance systems, as any breach can lead to long-term privacy, legal, and operational consequences.

1.1 Problem Statement

Biometric attendance systems improve operational efficiency by automating attendance tracking, but they also pose serious security risks due to their architectural design. Most existing systems rely on centralized cloud storage, which exposes sensitive biometric and attendance data to potential data breaches and unauthorized access.

The major security challenges in existing biometric attendance systems are:

- Reliance on centralized cloud infrastructure increases the risk of large-scale data breaches.
- Transmission of attendance data from devices to the cloud through insecure API endpoints.
- Susceptibility to Man-in-the-Middle (MitM) attacks and request tampering using tools such as cURL.

● Lack of strong verification mechanisms to validate the authenticity of data sources. These vulnerabilities make biometric attendance systems unsafe for handling sensitive data. Since biometric information is permanent and cannot be changed if compromised, any breach can have serious long-term effects. Therefore, it is essential to analyze these weaknesses and design a secure and reliable system that ensures data integrity, continuous availability, and protection against cyber threats.

1.2 Objectives

The primary objective of this project is to analyse security vulnerabilities in biometric attendance systems and to design a secure and reliable solution that can be applied across all domains.

The specific objectives of the project are as follows:

- To study the architecture and working of existing biometric attendance systems.
- To identify security vulnerabilities related to centralized cloud storage and insecure data transmission.
- To analyze risks such as Man-in-the-Middle attacks, request tampering, and unauthorized access.
- To examine the impact of continuous internet dependency on system reliability and data availability .
- To design a secure hybrid architecture using a local gateway for device validation and data encryption.
- To ensure data integrity, confidentiality, and system availability during network failures.
- To improve the overall security, scalability, and reliability of biometric attendance systems.

1.3 Existing Models

Existing biometric attendance systems are primarily designed to automate attendance recording by directly connecting biometric devices to centralized servers or cloud platforms. These models focus on convenience and efficiency but give limited importance to security, privacy, and reliability. Most systems use legacy biometric hardware combined with cloud-based data storage and processing.

Common characteristics of existing biometric attendance models include:

- Biometric devices directly transmit attendance data to a centralized cloud server.
- Use of legacy hardware with limited or no built-in security features.
- Data transmission often occurs over insecure or weakly protected API endpoints.
- Centralized data storage acts as a single point of failure.

Limitations of existing models:

- Sensitive biometric and attendance data are exposed to cyber threats such as interception and tampering.
- Vulnerable to Man-in-the-Middle attacks and replay attacks due to unencrypted communication.
- Heavy dependence on constant internet connectivity.
- Attendance data may be lost or delayed during network outages.

Due to these limitations, existing biometric attendance models are not suitable for securely handling sensitive biometric data in modern environments. These shortcomings highlight the need for a more secure, reliable, and scalable approach that addresses both security and operational challenges.

1.4 Proposed Model

The proposed model introduces a secure and reliable hybrid architecture to overcome the security and reliability issues present in existing biometric attendance systems. Instead of allowing biometric devices to communicate directly with centralized cloud servers, the proposed system places a local secure gateway between the biometric device and the cloud.

Key features of the proposed model include:

- A local secure gateway that acts as an intermediary between the biometric device and the cloud.
- Validation of biometric devices to ensure data is received only from authorized sources.
- Encryption of attendance data at the local level before transmission.
- Secure and controlled synchronization of encrypted data to the cloud.
- Reduced dependency on continuous internet connectivity by supporting local buffering.

CHAPTER 2
LITERATURE SURVEY

Biometric attendance systems have traditionally focused on automation and operational efficiency, relying primarily on fingerprint or facial recognition mechanisms for identity verification. However, the security of communication between biometric devices and centralized servers has received comparatively less attention. Many deployed systems transmit attendance logs over unsecured HTTP protocols, making them vulnerable to interception, replay attacks, and request tampering. Recent research therefore emphasizes the importance of secure communication protocols, encryption standards, and hybrid cloud–edge architectures to mitigate these risks. Several studies have proposed cryptographic and architectural solutions to protect sensitive biometric data during transmission and storage.

In [1], Galbally et al. analysed the resilience of biometric authentication systems against random inputs and demonstrated that certain biometric systems can be bypassed despite reporting high accuracy rates. The study highlighted that performance metrics alone do not guarantee security and that hidden vulnerabilities may exist in biometric verification systems. This work reinforced the need to strengthen security layers beyond basic authentication mechanisms in biometric attendance systems.

Complementing this, research in IoT security such as [3] by Abomhara and Køien provides a comprehensive overview of vulnerabilities in IoT-based systems, including data interception, replay attacks, and denial-of-service attacks. Since biometric attendance devices function as IoT endpoints, this study clarified that insecure network communication is a major threat vector. It emphasized the necessity of secure data transmission mechanisms and replay mitigation strategies in connected systems.

Addressing architectural weaknesses, Alrawais et al. in [2] proposed a secure hybrid cloud–edge architecture for IoT environments. The paper demonstrated that fully centralized cloud models introduce single points of failure and increase exposure to cyberattacks. By distributing processing and security tasks to edge devices, system resilience can be significantly improved. This directly motivates the proposed Raspberry Pi-based secure gateway model, which encrypts attendance data locally before cloud synchronization.

From a cryptographic perspective, the NIST Special Publication 800-38A [5] defines secure block cipher modes such as AES-CBC and provides guidelines for strong encryption standards. This foundational work supports the use of AES-256 encryption in the proposed system to ensure confidentiality of biometric attendance logs during transmission.

Biometric data protection has also been widely studied in template security research. Jain et al. in [4] discussed the risks associated with biometric template leakage and emphasized that biometric identifiers, unlike passwords, cannot be easily changed once compromised. This research highlights the importance of implementing encryption and secure storage mechanisms to protect sensitive student biometric data.

Beyond security, the importance of accurate attendance data for academic performance analysis is supported by educational research such as [6] by Freeman et al., which demonstrated a strong correlation between attendance and student outcomes. This underscores the need for attendance systems that are not only automated but also secure and reliable, as manipulated or compromised attendance data can directly impact academic evaluation.

Overall, the literature shows a clear progression from basic biometric authentication mechanisms toward secure, hybrid, and encryption-driven architectures capable of mitigating modern cybersecurity threats. These studies collectively motivate the proposed system, which integrates AES-256 encryption, timestamp-based replay prevention, gateway authentication using MAC address and API key validation, and a hybrid cloud–edge framework to ensure secure, reliable, and scalable biometric attendance management.

CHAPTER 3
THEORETICAL BACKGROUND

3.1 Biometric Systems and Secure Communication

3.1.1 What is a Biometric System

A biometric system is an automated identification system that verifies a person using unique physical or behavioural characteristics such as fingerprints, facial recognition, iris patterns, or voice recognition. These characteristics are unique to each individual and cannot be easily duplicated.

In attendance systems, biometric devices capture the user's identity and generate digital logs that include user ID, timestamp, and verification mode. While biometric systems increase automation and reduce manual errors, they handle highly sensitive personal data. If this data is exposed or misused, it can lead to serious privacy and security risks. Therefore, securing biometric systems is extremely important.

3.1.2 Biometric Attendance Systems

Biometric attendance systems are widely used in schools, colleges, and offices to automatically record presence. They eliminate manual roll calls and reduce proxy attendance or false entries. These systems improve efficiency and ensure accurate tracking of attendance records.

However, many legacy biometric devices focus only on identification and ignore communication security. In most cases, attendance data is transmitted over local networks without encryption. This makes the system vulnerable to data interception, replay attacks, and unauthorized manipulation. Hence, secure communication mechanisms are required.

3.1.3 Communication in Biometric Devices

Biometric machines typically communicate with servers using HTTP-based APIs. In many traditional systems, data is sent over Port 80 using plain text communication. This means anyone connected to the same network can view or capture the transmitted data. Such insecure communication exposes user IDs, timestamps, and punch types to network-level attacks like packet sniffing and request forging.

To protect sensitive attendance data, secure communication protocols such as HTTPS and encrypted payload transmission must be implemented.

3.2 IoT Architecture in Biometric Systems

3.2.1 IoT-Based Device Communication

Biometric machines act as IoT (Internet of Things) devices connected to institutional networks. IoT systems generally consist of three layers: edge devices (biometric machines), gateways, and cloud servers. These components communicate to collect, process, and store data.

Without proper security controls, IoT systems are highly vulnerable to cyber threats such as data interception, replay attacks, and denial-of-service attacks. Since biometric devices continuously transmit sensitive information, securing IoT communication channels becomes essential.

3.2.2 Hybrid Cloud–Edge Architecture

A hybrid cloud–edge architecture divides processing tasks between local devices (edge) and centralized cloud servers. In this model, sensitive data is first processed and encrypted locally before being sent to the cloud.

This approach reduces the risk of exposing raw data over the network and improves system reliability. Even if the internet connection fails, local gateways can temporarily store data. In the proposed system, a Raspberry Pi acts as a secure local gateway to implement this hybrid architecture.

3.3 Encryption and Cryptographic Security

Encryption is the process of converting readable data (plaintext) into an unreadable format (ciphertext) to prevent unauthorized access. Only authorized systems possessing the correct secret key can decrypt the ciphertext back into readable form. In security-sensitive systems such as biometric attendance management, encryption ensures confidentiality and protects sensitive data during transmission and storage.

In this project, AES-256 (Advanced Encryption Standard with 256-bit key length) is used to encrypt attendance logs before transmitting them to the cloud server. AES is a symmetric key encryption algorithm, meaning the same secret key is used for both encryption and decryption.

It is widely adopted globally and approved by organizations such as the National Institute of Standards and Technology (NIST) for securing classified and sensitive data.

AES-256 provides a 256-bit key length, which means there are 2^{256} possible key combinations. This extremely large key space makes brute-force attacks computationally infeasible with current technology. Compared to shorter key lengths such as AES-128, AES-256 offers a higher security margin, making it suitable for protecting highly sensitive biometric data.

In this system, AES is implemented in CBC (Cipher Block Chaining) mode. In CBC mode, each block of plaintext is XORed with the previous ciphertext block before encryption. Additionally, a random Initialization Vector (IV) is generated for each encryption process. This ensures that even if the same attendance data is encrypted multiple times, the resulting ciphertext will be different each time. This prevents pattern recognition and enhances security.

AES-256 was chosen over other cryptographic methods for the following reasons:

1. **Stronger than Basic Hashing (e.g., SHA-256):**

Hashing algorithms like SHA-256 are one-way functions and cannot be reversed. In this project, the server must decrypt attendance data for processing and storage. Therefore, encryption (reversible) is required instead of hashing.

2. **More Efficient than RSA for Bulk Data:**

RSA is an asymmetric encryption algorithm that is computationally intensive and slower for large data volumes. Since the Raspberry Pi gateway must handle multiple attendance records in real time, AES (symmetric encryption) provides much faster performance with lower computational overhead.

3. **Industry Standard and Widely Trusted:**

AES is widely used in secure communication protocols such as HTTPS, VPNs, and secure storage systems. Its reliability and strong mathematical foundation make it suitable for protecting biometric attendance data.

By implementing AES-256 encryption at the gateway level, the system ensures that attendance logs are encrypted before leaving the local network. Even if network traffic is intercepted, the attacker cannot read or manipulate the encrypted data without the secret key.

Algorithm	Type	Speed	Suitable for Bulk Data	Reversible	Security Level	Suitability for This Project
AES-256	Symmetric Encryption	Very Fast	Yes	Yes	Very High	Best choice – Strong security with high performance
AES-128	Symmetric Encryption	Very Fast	Yes	Yes	High	Good, but lower security margin than AES-256
RSA	Asymmetric Encryption	Slow	No	Yes	Very High	Too slow for real-time gateway encryption
SHA-256	Hashing Algorithm	Fast	No	No (One - Way)	High	Cannot decrypt data, unsuitable for transmission
DES	Symmetric Encryption	Moderate	Yes	Yes	Low	Outdated and insecure
3DES	Symmetric Encryption	Slow	Yes	Yes	Medium	Slower and less secure than AES

Table 3.3.1: Comparison of AES-256 with Other Cryptographic Algorithms

3.4 Replay Attacks and Mitigation

A replay attack occurs when an attacker captures a valid data packet and resends it later to manipulate the system. In biometric attendance systems, this could allow marking an absent student as present by resending a previously captured request.

To prevent replay attacks, timestamp validation is implemented to check whether the request is within an allowed time window. Additionally, database-level unique constraints prevent duplicate attendance records from being stored. These mechanisms ensure attendance integrity.

3.5 Zero Trust Security Model

The Zero Trust security model follows the principle “Never Trust, Always Verify.” It assumes that no device or request should be trusted automatically, even if it comes from inside the network.

In this project, every gateway request must be verified using a unique MAC address and API key. Only registered and active gateways are allowed to send attendance data. This approach minimizes the risk of unauthorized access and strengthens overall system security.

3.6 Database Integrity and Constraints

Database integrity ensures that stored data remains accurate, consistent, and reliable. In attendance systems, maintaining data accuracy is critical because it directly affects academic records and analytics.

The use of unique constraints (biometric_user_id, machine, timestamp) prevents duplicate entries in the database. This ensures that replayed or duplicate attendance logs are automatically rejected. As a result, the system maintains trustworthy and tamper-proof attendance records.

3.7 Human-in-the-Loop Principle

The Human-in-the-Loop principle means that the AI system helps teachers make decisions, but never makes decisions on its own. When the system generates a risk score or sends an alert about a student, a teacher or counsellor must review it first before any action is taken. No student is penalised or has their attendance record changed automatically by the system. A human always has the final say.

3.8 Algorithmic Bias and Fairness

Algorithmic bias occurs when a machine learning model produces results that systematically disadvantage certain groups due to patterns learned from biased or incomplete training data. In attendance-based predictive systems, bias can arise if the model is trained on data that includes demographic attributes such as gender, religion, or socioeconomic background, causing it to flag students unfairly based on identity rather than behaviour.

To prevent this, the predictive engine in the proposed system is trained exclusively on behavioural features including attendance frequency, consistency, timing patterns, and absence trends. All sensitive demographic identifiers are removed from the training dataset before model development.

3.9 Artificial Intelligence in the Proposed Attendance System

3.9.1 AI-Based Risk Assessment Model

In the proposed system, Artificial Intelligence is used to analyse attendance patterns and generate automated risk-based alerts. The AI component does not perform biometric recognition itself; instead, it operates at the cloud intelligence layer after attendance data is securely collected and synchronized.

The AI model evaluates factors such as:

- Missed check-in beyond threshold time
- Historical attendance behaviour
- Frequency of absences
- Time-of-day patterns
- Repeated late arrivals

Based on these parameters, the system computes a risk score for each student.

3.9.2 Score Generation Mechanism

After encrypted attendance logs are synchronized to the cloud server, the predictive analytics engine processes the data. The system runs a scheduled safety check (e.g., daily at 9:30 AM) to identify students who have not checked in.

The AI model then:

1. Retrieves attendance records from the database
2. Compares them with predefined threshold rules
3. Applies scoring logic based on absence frequency and behaviour
4. Generates a categorized risk level (Low / Medium / High)

This risk score determines whether an automated alert should be triggered.

3.9.3 Predictive Analytics Engine

The predictive engine functions as a rule-based and pattern-analysis module. It analyses both real-time attendance and historical data to detect anomalies.

For example:

- A student who is usually regular but misses one check-in may receive a low risk score.
- A student with repeated absences may receive a medium or high risk classification. This approach enables intelligent filtering instead of triggering alerts for every minor delay.

3.9.4 Automated Alert Generation

Once the risk score crosses a predefined threshold, the system automatically generates a security notification.

The notification includes:

- Student Name
- Student ID
- Date and Time
- AI Risk Assessment Level

Safety message for verification The alert is then sent to:

- Faculty
- Administrative authorities

This ensures proactive intervention rather than passive record-keeping.

3.9.5 Scheduler-Based AI Trigger

The AI workflow is activated by a scheduler module. For example, every school day at 9:30 AM:

1. The scheduler triggers the “Daily Safety Check”
2. The AI model evaluates missing attendance records
3. Risk scores are generated
4. Alerts are dispatched automatically

This ensures consistent and automated monitoring without manual intervention.

3.9.6 Security Integration with AI

Unlike traditional attendance systems, the AI engine operates only on securely transmitted and validated data. Before AI processing:

- Data is encrypted using AES-256
- Gateway authentication is verified
- Replay attacks are prevented
- Duplicate records are filtered

This ensures that AI decisions are made on authentic and untampered attendance logs.

3.9.7 AI Explainability in Attendance Analytics

Explainability refers to the ability of an AI or machine learning system to provide clear and understandable reasons for its predictions. In the proposed system, attendance analytics are designed to be interpretable so that the factors influencing risk classification can be easily understood.

Unlike complex deep learning models, which often act as black boxes and do not provide clear reasoning behind predictions, the system focuses on transparent analytical methods based on attendance features such as punctuality, attendance consistency, and consecutive absences. These features directly contribute to the calculation of performance scores and risk levels.For instance, the system can identify that an employee is marked as high risk due to frequent late entries or multiple consecutive absences. This transparency ensures that the system's outputs are explainable and verifiable.

CHAPTER 4
REQUIREMENT SPECIFICATIONS

4.1 User Requirements

- **Secure Attendance Recording**

The system must accurately record attendance using biometric authentication.

It should ensure that only genuine user punches are accepted and stored securely.

- **Secure Data Transmission**

Attendance data must be encrypted before being transmitted to the cloud server.

This prevents unauthorized access and protects sensitive biometric information.

- **Gateway Authentication**

Only registered and authorized gateway devices should be allowed to send data.

This ensures Zero Trust security and prevents unauthorized device access.

- **Real-Time Monitoring Dashboard**

Administrators should be able to view attendance logs and gateway status in real time.

This helps monitor system performance and detect anomalies quickly.

- **Replay Attack Prevention**

The system must detect and reject duplicate or delayed requests.

This ensures attendance integrity and prevents manipulation.

- **Automated Alert Generation**

The system must automatically send notifications when the AI detects high-risk cases.

This enables proactive monitoring and timely intervention.

4.2 Software Requirements

- **Programming Language -** Python

  Python is used for gateway development and backend implementation. It supports encryption, API development, and data processing.

- **Web Framework -** Django and Django REST Framework

  Django and Django REST Framework are used to build secure APIs. They enable structured request validation and database management.

- **Encryption Library -** PyCryptodome

  PyCryptodome is used for implementing AES-256 encryption.

It ensures secure cryptographic operations for data confidentiality.

- **Database System -** PostgreSQL / SQLite

  PostgreSQL / SQLite is used for storing attendance logs and gateway data. It ensures structured storage and supports integrity constraints.

- **Development Tools -** Notepad++ / VS Code and Postman

  Notepad++ / VS Code and Postman are used for development and API testing. These tools assist in debugging, validation, and system verification.

4.3 Hardware Requirements

- **Biometric Device -** eSSL biometric machine

An eSSL biometric machine is used for capturing attendance data.

It records user ID, timestamp, and verification mode.

- **Raspberry Pi Gateway**

A Raspberry Pi is used as a secure local gateway device.

It encrypts attendance data and forwards it securely to the cloud.

- **Network Infrastructure**

A stable LAN/Wi-Fi network is required for device communication.

It enables secure data transfer between biometric devices and the server.

CHAPTER 5

DESIGN AND IMPLEMENTATION

5.1 Methodology

The proposed methodology follows a hybrid cloud–edge architecture to enhance the security and reliability of biometric attendance systems. Instead of sending attendance data directly to the cloud, a Local Secure Gateway is introduced between the biometric device and the server. This gateway validates device identity, encrypts attendance logs, and ensures controlled communication. The approach reduces exposure to network-level threats and strengthens overall system security.

After biometric authentication, attendance data is encrypted using AES-256 and appended with a timestamp to prevent replay attacks. If internet connectivity is unavailable, the encrypted records are securely stored in a local buffer and automatically synchronized when the connection is restored. The cloud server then verifies, decrypts, and stores validated records in the database. This methodology ensures secure transmission, zero data loss, and reliable attendance management.

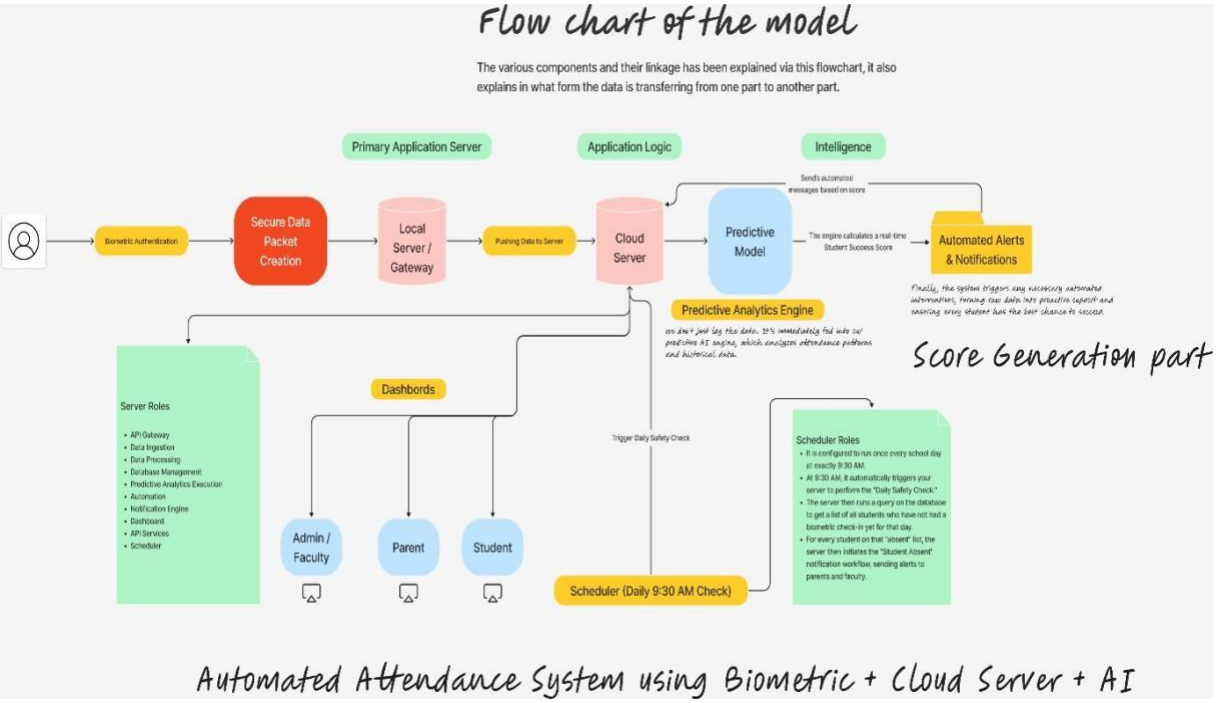


Figure 5.1: AI-Integrated Biometric Attendance System – Complete Workflow Architecture

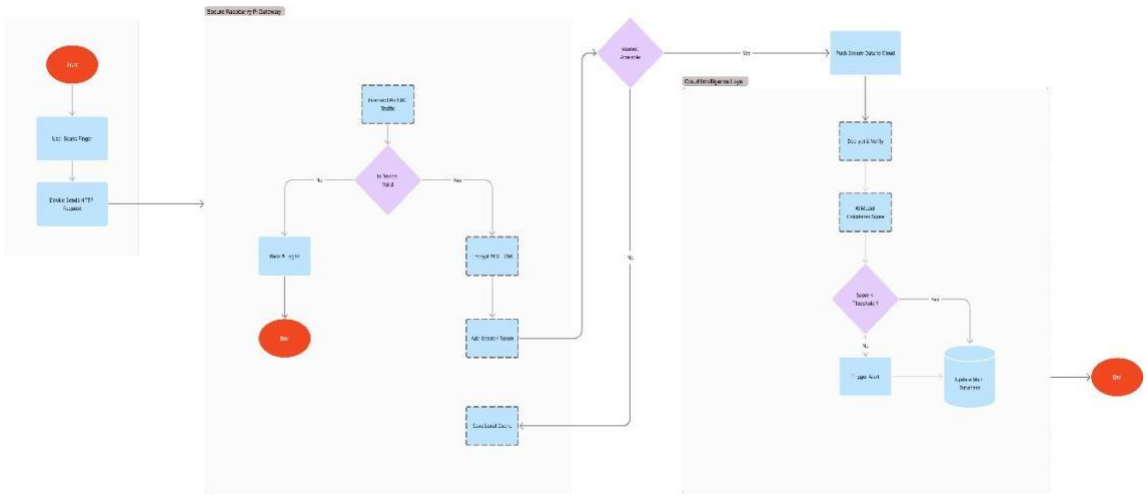


Figure 5.2: AI-Integrated Biometric Attendance System – Flow Chart

Step 1: Start

The process begins when the biometric attendance system is powered on and ready to record attendance. The system waits for user interaction. All components including the biometric device and gateway remain active for real-time processing. This marks the initialization stage of the secure attendance workflow.

Step 2: User Scans Fingerprint

The user places their fingerprint on the biometric scanner. The device verifies the fingerprint locally and generates attendance data containing user ID, timestamp, and punch type. This raw attendance record is prepared for transmission. At this stage, the data is still unencrypted.

Step 3: Device Sends HTTP Request

The biometric device sends the attendance data as an HTTP request over Port 80. In traditional systems, this communication is unencrypted and vulnerable to interception. The request contains attendance details in plain text format. This creates a potential security risk if not protected.

Step 4: Data Goes to Local Gateway

Instead of sending the data directly to the cloud server, the HTTP request is routed to the Raspberry Pi local gateway. The gateway acts as a middleware security layer between the biometric device and the cloud. This ensures all traffic passes through a secure checkpoint.

Step 5: Intercept Port 80 Traffic

The local gateway intercepts the incoming Port 80 HTTP traffic. It monitors and captures the attendance payload before it leaves the local network. This prevents insecure direct communication with the cloud. It allows the gateway to apply security controls before forwarding data.

Step 6: Check if Device is Valid

The gateway verifies whether the request is coming from an authorized biometric device. It checks device identity using predefined credentials such as MAC address or device serial number.

If the device is not valid, the request is rejected and the process ends. If valid, the system continues securely. This step ensures only trusted devices can send attendance data.

Step 7: Encrypt Using AES-256 and Add Session Token

Once validated, the gateway encrypts the attendance data using AES-256 encryption. A session token and timestamp are attached to the encrypted payload to prevent replay attacks and ensure secure session management. This protects the confidentiality and integrity of the attendance data.

Step 8: Check Internet Connectivity

The gateway checks whether internet connectivity is available. This ensures reliable data transmission without losing attendance records. The system adapts based on network availability to maintain operational continuity.

Step 9: Save in Local Cache (If No Internet)

If the network connection is unavailable, the encrypted attendance logs are stored securely in local memory or cache. This prevents data loss during network failure. Once connectivity is restored, the cached data is automatically synchronized with the cloud.

Step 10: Push Data Securely to Cloud

If internet connectivity is available, the encrypted attendance payload is securely transmitted to the cloud server using an HTTP POST request. Since the data is already encrypted, sensitive

information remains protected during transmission. This ensures safe edge-to-cloud communication.

Step 11: Cloud Intelligence Layer – Decrypt and Verify

At the cloud layer, the server verifies the gateway using MAC address and API key validation under the Zero Trust model. After successful authentication, the server decrypts the AES-256 encrypted payload. The attendance data is validated, checked for duplicates, and stored securely in the database. This completes the secure attendance processing cycle.

Step 12: AI Model – Attendance Pattern Analysis

After successful decryption and validation, the processed attendance data is forwarded to the AI analytics module. The AI model analyses attendance patterns, frequency of absences, and timing irregularities using predefined rules or predictive logic. This helps in identifying abnormal behaviour or risk patterns. The system converts raw attendance logs into meaningful analytical insights.

Step 13: Risk Score Calculation

The AI engine calculates a risk score based on attendance consistency, missed check-ins, and predefined thresholds. The score represents the probability of irregular attendance or potential safety concern. This automated evaluation removes manual monitoring effort. The calculated score is then compared with the predefined threshold value.

Step 14: Threshold Evaluation and Decision

The system checks whether the generated AI score exceeds the configured risk threshold. If the score is below the threshold, the system simply updates the database without triggering alerts. If the score exceeds the threshold, it indicates potential concern. This conditional decision-making ensures alerts are generated only when necessary.

Step 15: Automated Alert and Notification

If the risk score crosses the threshold, the system automatically triggers a notification workflow. Alerts are sent to authorized stakeholders such as administrators, parents, or faculty members. The alert includes student details, timestamp, and AI risk assessment level. This ensures proactive monitoring and timely intervention.

5.1.1 Development Phases

The project was developed using a structured phase-wise approach to ensure proper implementation, testing, and validation.

Phase 1: Planning and Architecture

This phase focuses on designing the overall structure and communication flow of the system.

- Defined system architecture (Biometric Device → Raspberry Pi → Django Dashboard)
- Designed secure network flow and security boundaries
- Analyzed biometric device communication protocol

Phase 2: Raspberry Pi Secure Gateway (Edge Layer)

This phase involves developing the edge-level security layer for data processing and protection.

- Developed Python-based gateway system
- Implemented local SQLite database for offline buffering
- Applied AES/RSA encryption for secure data transmission
- Enabled secure synchronization with cloud server
- Configured auto-start and continuous execution services

Phase 3: Django Server & Dashboard (Cloud Layer)

This phase focuses on backend development and data visualization.

- Built backend using Django framework
- Designed database schema for users, devices, and attendance logs
- Developed secure REST APIs with JWT/API key authentication
- Implemented decryption logic for incoming data
- Created responsive dashboard for monitoring and analytics

Phase 4: Testing & Security Validation

This phase ensures system reliability, security, and performance.

- Tested device-to-gateway communication
- Validated offline buffering during network failures
- Performed security testing against MITM attacks
- Conducted end-to-end system testing from device scan to dashboard output

5.1.2 Security Framework (Zero-Trust)

The system follows a Zero-Trust security model where every request is verified before access is granted.

Challenge–Response Handshake

A three-factor authentication mechanism is used to validate devices:

- MAC Address: Unique network identifier of the device
- Hardware Serial: Ensures device authenticity and prevents spoofing
- One-Time Nonce: Prevents replay attacks using cryptographic validation

Dynamic Key Derivation

Encryption keys are generated dynamically for each session instead of being stored.

- Session keys are derived using PBKDF2 algorithm
- Uses server-side nonce as salt
- Uses API key as master secret
- Ensures secure and temporary encryption keys for each transmission

5.1.3 Local AI-Based Risk Analysis

The system performs AI-based behavioral analysis using a local language model (Llama 3.2 via Ollama), eliminating dependency on external APIs. Historical attendance data is analyzed to evaluate student behavior, including punctuality and consistency. Based on these metrics, the system assigns a risk level such as LOW, MEDIUM, or HIGH. This approach enables early identification of irregular attendance patterns and supports proactive intervention while maintaining data privacy and reducing latency.

5.1.4 Task Scheduling Mechanism

A task scheduling mechanism is implemented to automate routine operations such as daily attendance monitoring. The system uses a crontab-based scheduler to execute tasks at predefined times. The primary task, daily safety check, runs once every day at a fixed time. The system also supports interval-based execution and one-time scheduled tasks. All scheduled tasks are managed through a centralized periodic task system, ensuring flexibility, scalability, and efficient task management.

Class Diagram

The Use Case Diagram illustrates the interaction between users, devices, and system components in the secure biometric attendance system. It highlights the major functionalities such as attendance marking, encryption, validation, analytics, and dashboard monitoring.

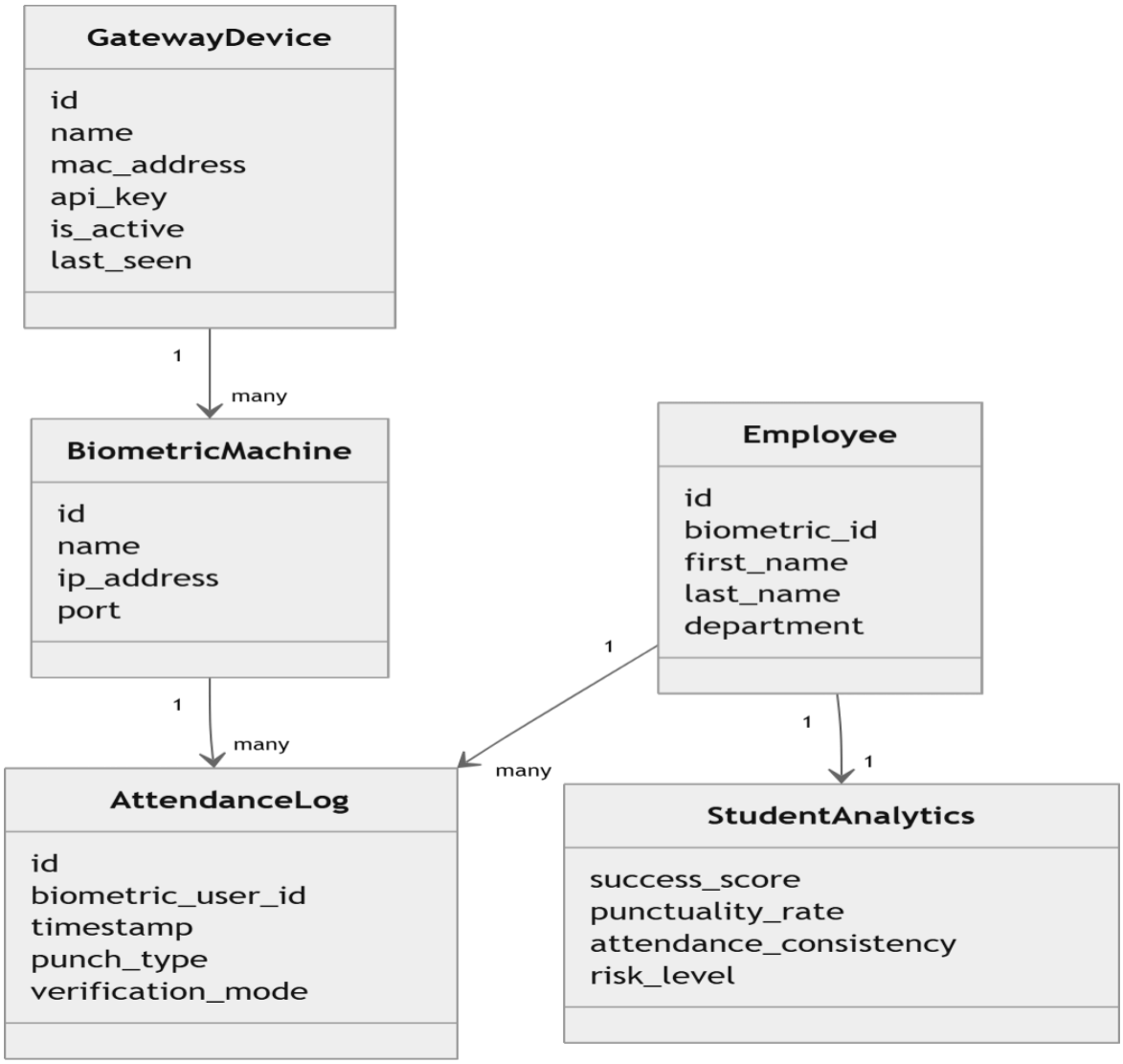


Figure 5.3: Class Diagram

Use case Diagram

The Use Case Diagram illustrates the interaction between users, devices, and system components in the secure biometric attendance system. It highlights the major functionalities such as attendance marking, encryption, validation, analytics, and dashboard monitoring.

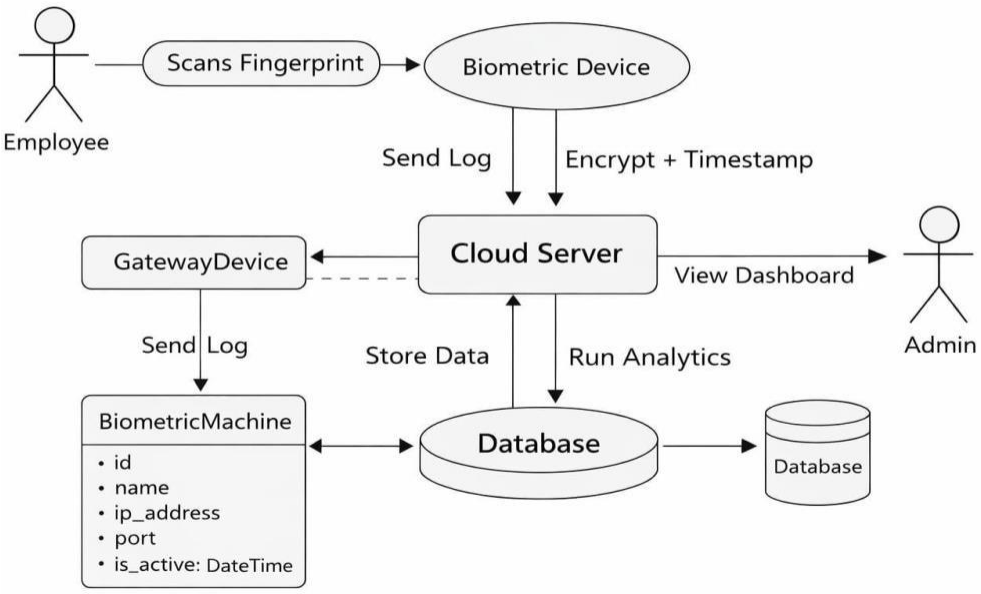


Figure 5.4: Use Case Diagram

Sequence Diagram

The Sequence Diagram demonstrates the chronological interaction between system components during attendance processing. It clearly outlines the secure communication flow, including encryption, validation, storage, and AI-based risk evaluation.

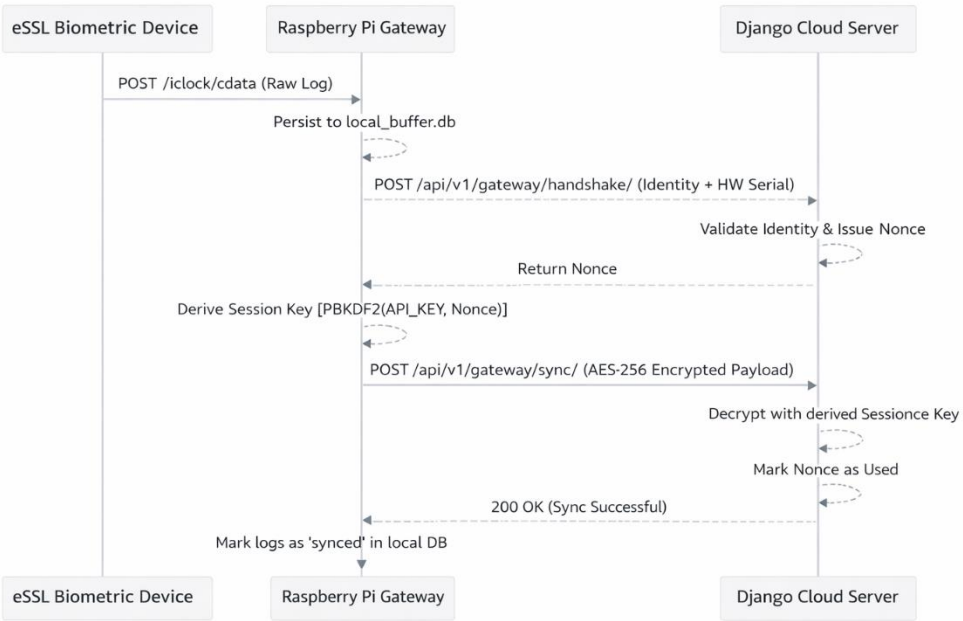


Figure 5.5: Sequence Diagram

Deployment Diagram

The Deployment diagram represents the physical architecture of the proposed hybrid system. It shows how biometric devices, gateway hardware, cloud servers, and databases are deployed and communicate to provide secure attendance processing.

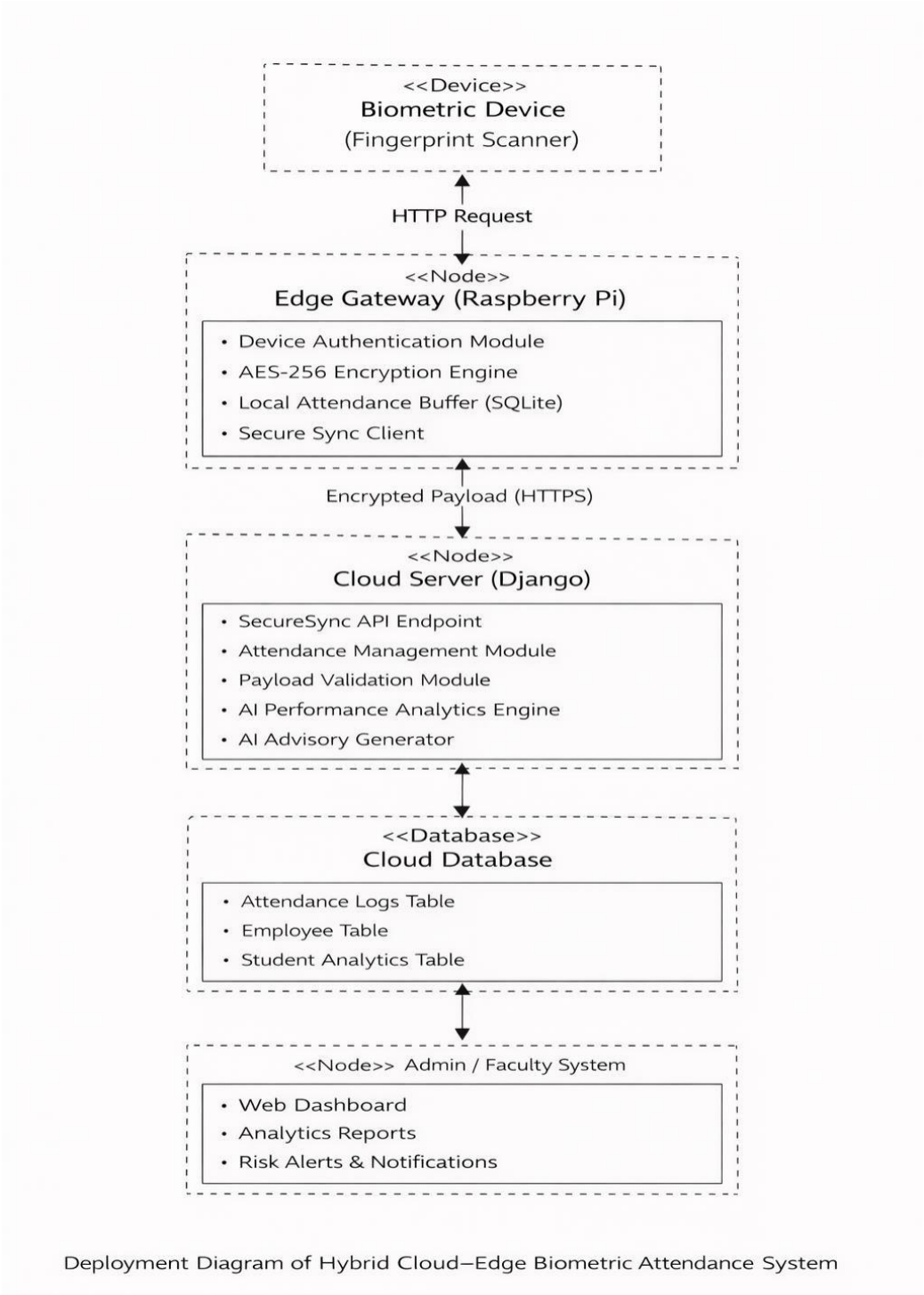


Figure 5.6: Deployment Diagram

5.2 Modules

The Secure Biometric Attendance System is organized into the following modules based on the architectural workflow shown in the design diagram:

1. Biometric Device Module

This module captures user attendance through fingerprint or facial recognition. It generates raw attendance logs containing user ID, timestamp, and punch type. The device sends this data as an HTTP request to the local gateway for secure processing.

2. Traffic Interception Module (Gateway Layer)

This module intercepts incoming HTTP traffic on Port 80 at the Raspberry Pi gateway. It ensures that attendance data does not directly reach the cloud server without security validation. This module acts as the first security checkpoint in the system.

3. Device Validation Module

This module verifies whether the biometric device sending the request is authorized. It checks predefined parameters such as MAC address or device identity. Unauthorized devices are rejected, enforcing Zero Trust security principles.

4. AES-256 Encryption Module

This module encrypts attendance logs using AES-256 encryption in CBC mode. A random Initialization Vector (IV) and session token are added before transmission. This ensures that attendance data remains confidential during network communication.

5. Offline Caching Module

This module checks internet connectivity before transmitting data to the cloud. If connectivity is unavailable, encrypted attendance logs are stored securely in local cache. Once the connection is restored, the cached data is synchronized automatically.

6. Secure Cloud Transmission Module

This module securely sends encrypted attendance data to the Django cloud server using a POST request. Since the data is encrypted at the gateway, it remains protected during transmission across public networks.

7. Cloud Authentication and Decryption Module

This module validates the gateway using MAC address and API key authentication. After successful verification, the server decrypts the AES-256 encrypted payload. Timestamp validation is performed to prevent replay attacks before processing.

8. Database Integrity Module

This module stores attendance logs securely in the database. Unique constraints are applied on biometric_user_id, machine, and timestamp to prevent duplicate entries. This ensures data accuracy and protects against replayed or manipulated requests.

9. Dashboard and Monitoring Module

This module provides a web-based dashboard for administrators. It displays attendance logs, gateway status, and real-time monitoring information. It enables secure visualization and system mana The system is built using a combination of modern technologies to ensure security, scalability, and performance.

5.2.1 Technical Stack

The system is built using a combination of modern technologies to ensure security, scalability, and performance.

- **Backend:** Django 5.x – Used as the core framework for handling business logic and server operations
- **REST API:** Django REST Framework (DRF) – Enables secure and structured data communication
- **Edge Gateway:** Flask (Python) – Lightweight server running on Raspberry Pi for device communication
- **VPN:** Tailscale – Provides secure remote access and network connectivity
- **Encryption:** AES-256-CBC – Ensures strong data protection during transmission
- **Key Exchange:** PBKDF2 Challenge-Response – Generates dynamic session-based encryption keys
- **Database (Cloud):** SQLite/PostgreSQL – Stores centralized attendance and user data

- **Database (Edge):** SQLite – Maintains local buffer for offline data storage
- **Task Queue:** Celery with Redis – Handles asynchronous tasks like email alerts and scheduling
- **AI Analytics:** Ollama (Llama 3) – Performs local AI-based student risk analysis
- **Alerts:** AWS SES – Used for sending automated email notifications

5.2.2 Database Schema

The system uses both cloud and edge databases for efficient data handling.

Cloud (Django)

- **GatewayDevice:** Stores authorized gateway devices with MAC, API key, and hardware serial
- **GatewaySession:** Manages active sessions and nonce validity
- **Employee:** Stores student/staff details including parent email
- **AttendanceLog:** Stores final synchronized attendance records
- **AlertSettings:** Configures alert timing and notification settings

Edge (Raspberry Pi)

- **attendance_logs:** Stores local attendance data including user ID, timestamp, device IP, punch type, and sync status

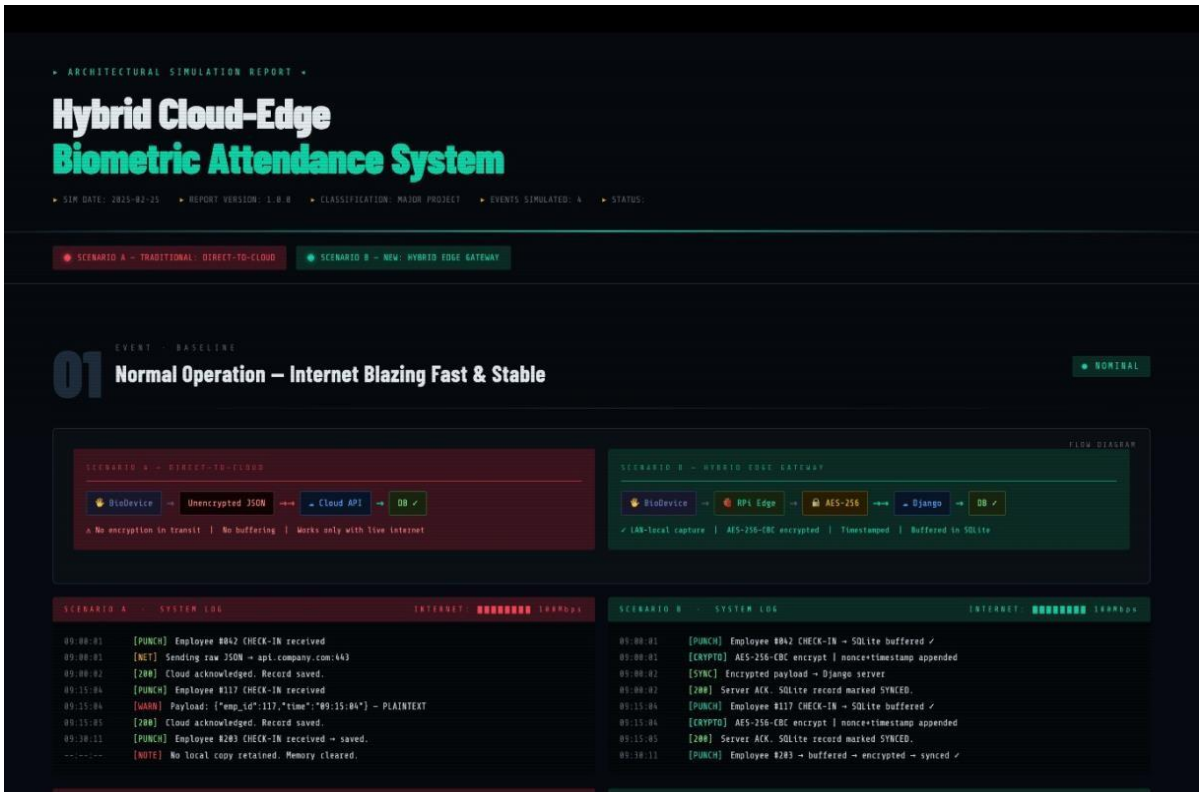


Figure 5.7: Baseline System Architecture – Traditional Direct-to-Cloud vs Hybrid Edge Gateway

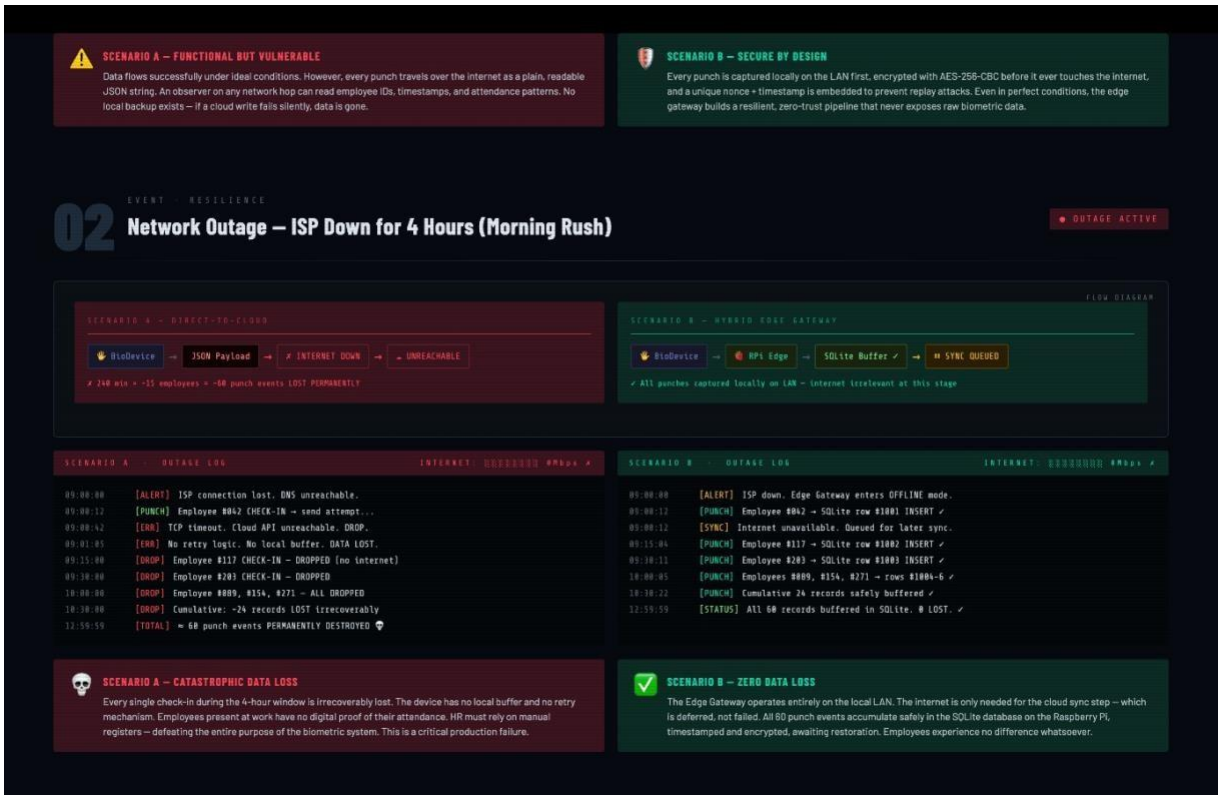


Figure 5.8: Network Outage Scenario – Data Loss vs Secure Local Buffering

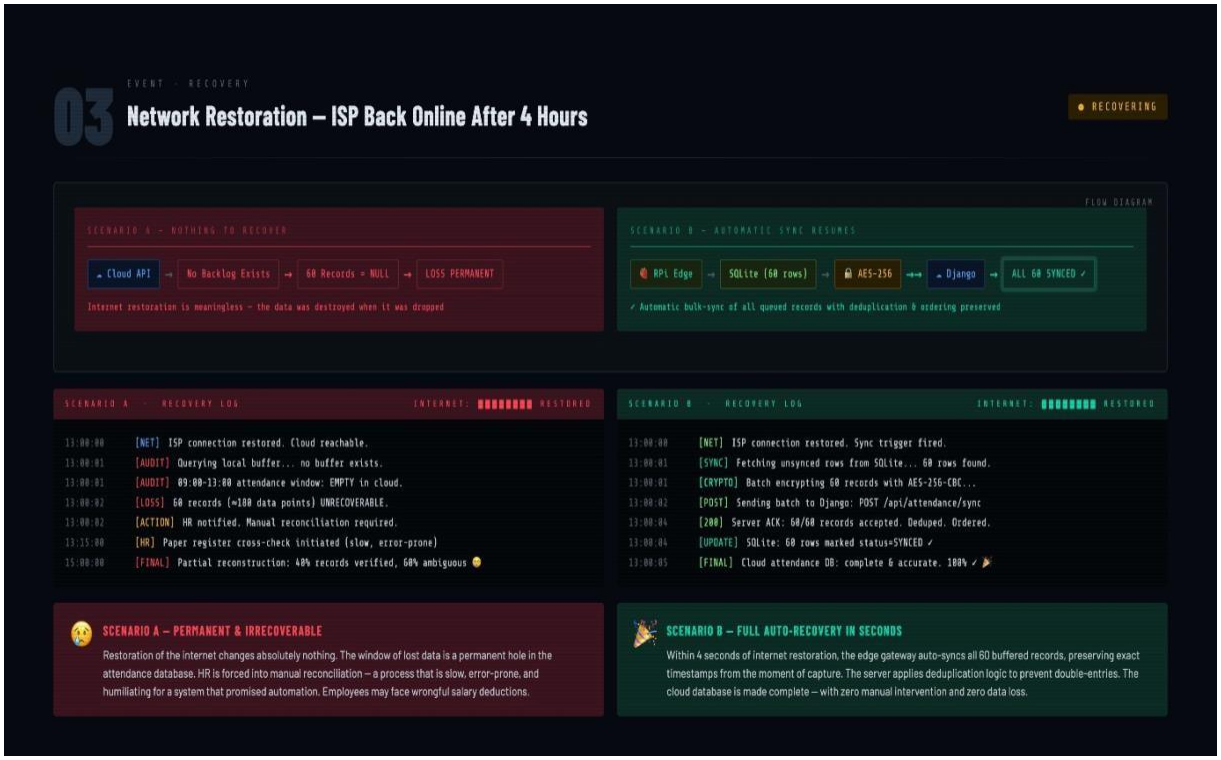


Figure 5.9: Network Restoration – Automatic Cloud Resynchronization

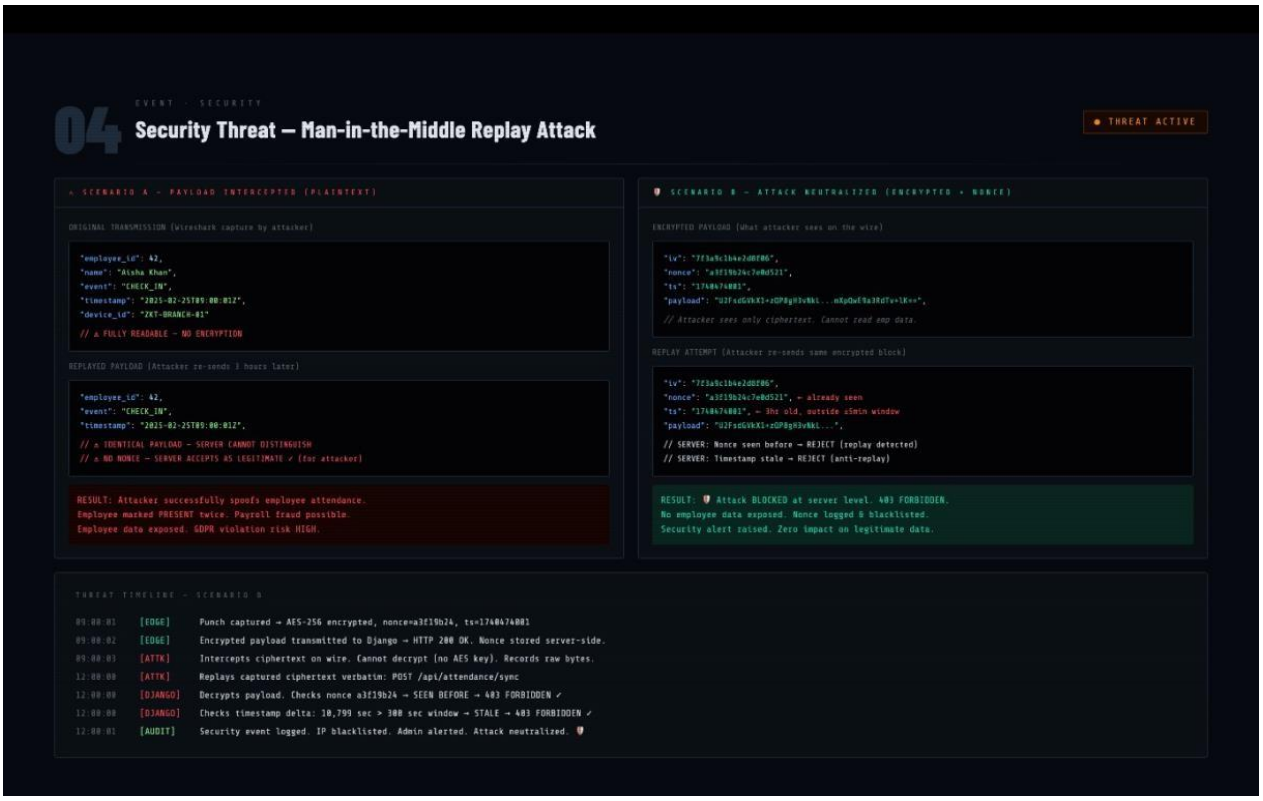


Figure 5.10: Replay Attack Simulation and Server-Side Prevention Mechanism

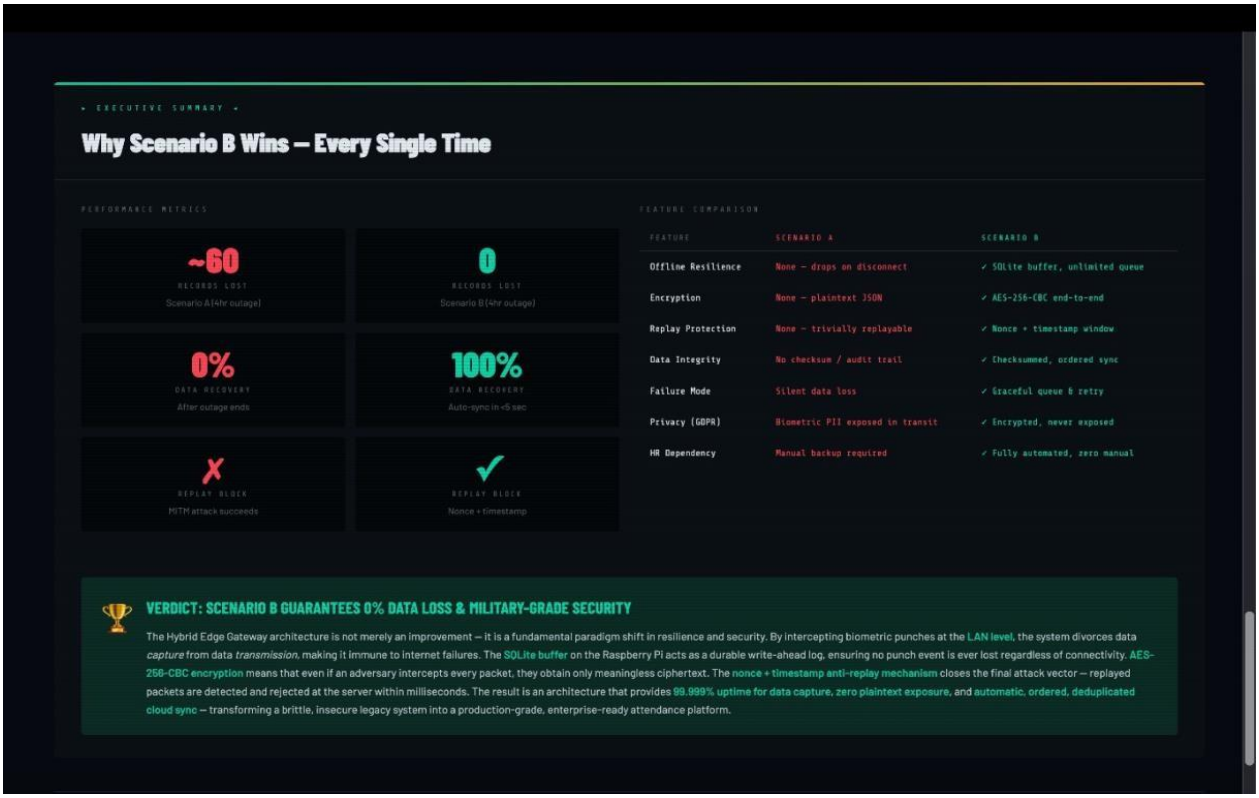


Figure 5.11: Comparative Performance and Security Analysis – Traditional vs Hybrid Edge Architecture

CHAPTER 6
DATA EXPLORATION AND ANALYSIS

This section describes the process of examining and analyzing the attendance data transmission and network communication patterns of the biometric system. The objective of data exploration in this project was not predictive modelling, but rather identifying security weaknesses in how attendance data is generated, transmitted, and stored.

6.1 Data Used

The system uses attendance data generated from biometric devices. Each record includes user ID, timestamp, punch type, and verification mode. In addition to attendance logs, network packet data captured using tools such as Wireshark is used to analyze vulnerabilities in data transmission. This data helps in identifying insecure communication patterns and validating the effectiveness of the proposed secure gateway model.

6.2 Data Collection

Attendance data is collected from the biometric device when a user scans their fingerprint or face. The device generates logs containing user ID, timestamp, punch type (check-in/check-out), and verification mode. These raw logs are transmitted to the secure gateway and later stored in the database after encryption and validation.

6.3 Data Storage and Structure

After secure transmission and decryption, attendance logs are stored in the database. Each record includes biometric_user_id, machine details, timestamp, punch_type, and verification_mode. Unique constraints are applied to prevent duplicate entries. The structured storage enables easy querying and analysis.

The database schema is designed to maintain relational integrity between employees, machines, and gateways. This structured approach ensures efficient retrieval of attendance records for reporting, monitoring, and analytical processing while maintaining data consistency and reliability.

6.4 Data Cleaning and Validation

Before analysis, the data undergoes multiple validation checks to ensure accuracy and reliability:

- Duplicate entries are removed using database-level unique constraints
- Invalid or outdated timestamps are rejected through replay attack protection mechanisms.
- Unauthorized device logs are filtered out using gateway authentication validation.
- Missing or unregistered employee records are handled appropriately to avoid inconsistent entries.

In addition to these checks, the system verifies the format and completeness of each attendance record before storage. Any corrupted or partially transmitted data packets are discarded to prevent inaccurate analysis. These validation steps ensure that only authentic, consistent, and verified attendance data is used for further statistical evaluation and predictive modelling.

6.5 Data Exploration

Data exploration involves understanding attendance patterns and trends. The following aspects are analyzed:

- Daily attendance count
- Number of active gateways
- Punch frequency distribution
- Peak attendance timings
- Missing attendance patterns

Exploratory queries help identify anomalies such as repeated entries, unusual time stamps, or inactive devices.

Date	Total Employees	Present	Absent	Attendance Rate
20-02-2026	120	108	12	90%
21-02-2026	120	110	10	91.6%
22-02-2026	120	110	15	87.5%

Table 6.4.1: Daily Attendance Summary

Explanation:

This table shows the daily attendance statistics obtained from the biometric attendance logs. It helps in understanding the participation level of employees and identifying variations in attendance patterns.

6.6 Feature Extraction

For advanced analysis and AI-based predictive modelling, meaningful features are extracted from attendance data. These features transform raw logs into structured inputs for evaluation:

- Total attendance percentage
- Number of absences
- Consistency of attendance
- Average arrival time
- Frequency of late entries
- These features form the basis for evaluating attendance behavior.

These extracted features serve as inputs for the AI module to assess attendance behavior. By converting attendance logs into measurable indicators, the system enables structured behavioral evaluation and risk scoring.

Time Range	Number of Punches	Percentage
08:45 – 09:00	42	35%
09:00 – 09:05	36	30%
09:05 – 09:15	28	23%
After 09:15	14	12%

Table 6.5.1: Punch Time Distribution

Explanation:

This table represents the distribution of check-in times recorded by the biometric system. The analysis helps identify peak attendance periods and detect late arrival patterns.

Employee ID	Consecutive Absences	Risk Indicator
EMP1001	0	Low
EMP1002	2	Medium
EMP1003	5	High

Table 6.5.2: Missing Attendance Pattern

Explanation:

This table shows employees with consecutive absences, which helps identify irregular attendance behaviour. These patterns are later used by the analytics engine to determine attendance risk levels.

6.7 Visualization and Dashboard Reporting

The analyzed data and AI-generated insights are displayed through a real-time dashboard interface. Administrators can monitor:

- Total attendance logs
- Active gateway status
- Attendance trends (daily, weekly, monthly)
- Late entry statistics
- Device performance metrics
- AI-generated risk scores
- Alert history and notifications

Interactive dashboards provide visual representations such as charts, tables, and trend indicators. This visualization supports informed decision-making, proactive monitoring, and improved institutional oversight.

CHAPTER 7
MODELLING AND IMPLEMENTATION

This section explains the development of the proposed secure hybrid model and evaluates its effectiveness in mitigating the vulnerabilities identified in existing biometric attendance systems. The modelling phase focused on designing a secure architecture that eliminates direct device-to-server communication and introduces validation and encryption at the edge level. The implementation phase ensured that the proposed design could practically prevent replay attacks, unauthorized access, and data manipulation.

7.1 Model Development

The Secure Biometric Attendance System model is developed to ensure protected data transmission, secure storage, and reliable attendance analytics. The primary objective of the model is to eliminate vulnerabilities in traditional biometric systems by introducing encryption, replay protection, and a hybrid cloud–edge architecture. The model ensures that sensitive biometric attendance data is never transmitted or stored in unsecured form.

System Architecture Model

The system follows a hybrid cloud–edge architecture. Biometric devices capture attendance data and send it to a Raspberry Pi local gateway instead of directly communicating with the cloud server. The gateway intercepts Port 80 traffic, validates the device identity, encrypts the data using AES-256, and forwards the encrypted payload to the cloud server.

This layered architecture integrates authentication, encryption, validation, secure transmission, decryption, and database storage into a unified secure workflow. The model ensures that raw attendance data is protected before leaving the local network.

Data Representation Model

Attendance data is represented in structured format containing biometric_user_id, timestamp, punch_type, and verification_mode. The data is converted into JSON format before encryption at the gateway.

This structured representation enables efficient encryption, secure transmission, and consistent database storage. It also supports future analytical processing and predictive modeling.

Encryption and Secure Transmission Model

AES-256 encryption in CBC mode is implemented at the gateway level to protect attendance logs during transmission. A random Initialization Vector (IV) is generated for each encryption process to enhance security. A timestamp and session token are attached to prevent replay attacks.

The model ensures that encryption is applied immediately after device validation, thereby minimizing exposure of sensitive data. The encrypted payload is transmitted to the cloud server using secure HTTP communication.

Authentication and Replay Protection Model

The model incorporates a Zero Trust authentication mechanism. Each gateway is uniquely identified using a MAC address and API key. The server validates these credentials before decrypting any data.

Additionally, timestamp validation and database-level unique constraints prevent replay attacks and duplicate attendance entries. This ensures integrity and reliability of attendance records.

Database Integrity Model

Attendance logs are stored in a structured relational database. A unique constraint on biometric_user_id, machine, and timestamp prevents duplicate entries.

This model guarantees accurate attendance tracking and protects against tampering or repeated log insertion. It also enables efficient querying and monitoring through the dashboard interface.

Analytical and Monitoring Model

The system includes a monitoring layer that analyzes attendance trends, daily logs, and gateway status. Statistical summaries such as attendance percentage and frequency patterns are generated for administrative review.

The analytical model supports identification of irregular attendance patterns and enables predictive analytics for early intervention in academic settings.

Email Notification System & Logging

A responsive HTML-based email notification system was implemented to enhance communication and alert visualization. The system replaces plain text emails with a structured and professionally styled format. Each email dynamically includes student details such as name, ID, and alert timestamp. Additionally, an AI-based Risk Assessment Badge is incorporated, which categorizes students into LOW, MEDIUM, and HIGH risk levels using color indicators.

To ensure transparency and traceability, a database model named NotificationLog was developed to record all outgoing emails. The system logs recipient email, recipient type (student/parent/faculty), timestamp, and delivery status (sent/failed). These logs are integrated into the admin dashboard with search and filtering capabilities, enabling easy monitoring and auditing of notifications.

Multi-Party Notification System

The system supports automated email notifications to multiple stakeholders simultaneously. When abnormal attendance patterns are detected, alerts are triggered and sent to students, parents, and faculty members. This ensures real-time awareness and facilitates coordinated action to address attendance issues effectively.

Simulation and Deployment

Developed simulation scripts to compare direct cloud communication and gateway-based architecture. Demonstrated that direct cloud systems suffer from data loss during network failures. The gateway-based system ensures zero data loss through local buffering and synchronization. The system was deployed on a Raspberry Pi with continuous execution using system services for reliability. Additionally, IoT-based remote management was enabled using VPN for secure monitoring and access.

AI Analytics and Notification System

Implemented an AI-based Student Analytics model to calculate a Success Score based on attendance behavior. Developed an automated daily safety check system to monitor attendance and identify irregular patterns. Integrated AWS SES for sending automated email notifications to students, parents, and faculty. Designed a dynamic alert configuration system that can be controlled through the admin panel. Implemented Celery Beat scheduler to automate periodic task execution efficiently.

7.2 Model Evaluation

The Secure Biometric Attendance model is evaluated based on correctness, security strength, system performance, and operational feasibility.

Functional Correctness

The evaluation confirms that encrypted attendance logs are successfully decrypted at the cloud server without data loss. Stored attendance records match the original device-generated logs after decryption. Duplicate prevention and timestamp validation mechanisms function correctly to maintain data integrity.

Security Evaluation

The system ensures that attendance data is never transmitted in plaintext form beyond the local gateway. AES-256 encryption protects data confidentiality during transmission. Gateway authentication and replay protection mechanisms reduce risks of unauthorized access and data manipulation. The Zero Trust model strengthens overall system security by validating every request before processing.

Security Evaluation

The system is scalable and can handle multiple biometric devices and large volumes of attendance data. The use of a gateway-based architecture reduces load on the cloud server, allowing the system to expand without significant performance degradation. Additional devices can be integrated easily without modifying the core system design.

Resilience Evaluation

The system demonstrates high resilience under adverse conditions such as network failures. The edge gateway stores attendance data locally and ensures automatic synchronization once connectivity is restored. This guarantees continuous system operation without data loss.

Performance Analysis

Encryption introduces minimal computational overhead at the gateway. AES-256 encryption is efficient enough to handle multiple attendance entries in real time. The hybrid architecture ensures that encryption processing does not overload the central cloud server. Offline caching ensures system continuity during network outages, maintaining operational resilience.

AI Model Effectiveness

The AI-based analysis module effectively evaluates attendance patterns and assigns appropriate risk levels. The system identifies irregular attendance behavior such as inconsistency and late arrivals. The generated risk levels (Low, Medium, High) help administrators take timely actions. The use of a local AI model ensures faster processing.

Maintainability

The system is modular in design, allowing easy updates and enhancements. Components such as the gateway, server, and AI module can be modified independently without affecting the entire system. This simplifies debugging, maintenance, and future upgrades.

Usability and Practical Feasibility

The system provides a user-friendly dashboard for administrators to monitor attendance logs and gateway status. The encryption and validation processes operate transparently in the background without affecting user interaction. The use of Raspberry Pi as a local gateway demonstrates cost-effective implementation, making the model practical for educational institutions.

The evaluation results confirm that the proposed model successfully enhances security in biometric attendance systems while maintaining operational efficiency and scalability. The system effectively balances data protection, performance, and usability.

CHAPTER 8
RESULTS AND CONCLUSIONS

8.1 Results

The implementation of the Local Secure Gateway significantly improved the security and reliability of the biometric attendance system. During the initial analysis phase, it was confirmed that legacy systems transmitted attendance data in plain-text format using HTTP. This allowed sensitive information such as user IDs, timestamps, and device serial numbers to be intercepted and manipulated.

After integrating the secure gateway, all attendance logs were intercepted, validated, and encrypted before being forwarded to the central server. Replay attempts using previously captured requests were successfully detected and rejected. Unauthorized device access was also blocked, ensuring that only registered devices could transmit attendance data.

The system was further tested under simulated network failure conditions. The gateway temporarily stored attendance logs locally and synchronized them automatically once internet connectivity was restored. This prevented data loss and ensured continuous system operation.

S. No	Test Scenario	Existing System Behaviour	Proposed System Behaviour	Result Status
1	Plain HTTP Transmission	Data visible in plain text	Data encrypted before transmission	Secured
2	Replay Attack Attempt	Attendance accepted without verification	Unauthorized replay rejected by gateway	Prevented
3	Unauthorized Device Access	Server accepts formatted request	Gateway blocks unregistered device	Blocked
4	Data Integrity Check	No integrity validation	Validated before storage	Verified
5	Network Failure	Data loss possible	Local buffering and later synchronization	Reliable
6	Device Authentication	No strict device verification	Device identity validated at gateway	Authenticated

Table 8.1.1: Performance and Security Evaluation of Proposed System

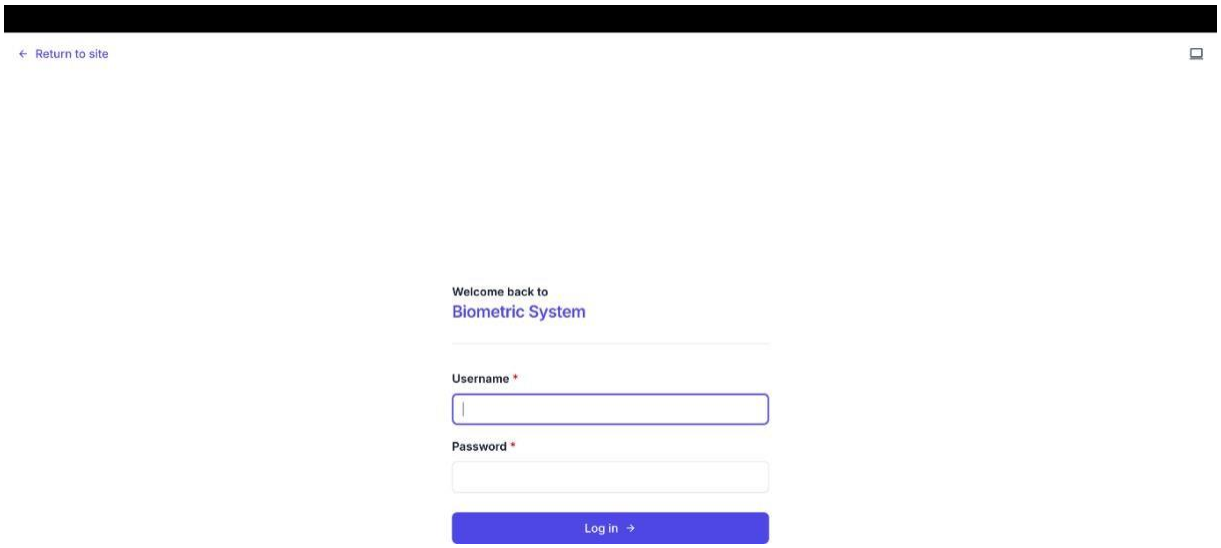


Figure 8.1: Login Interface

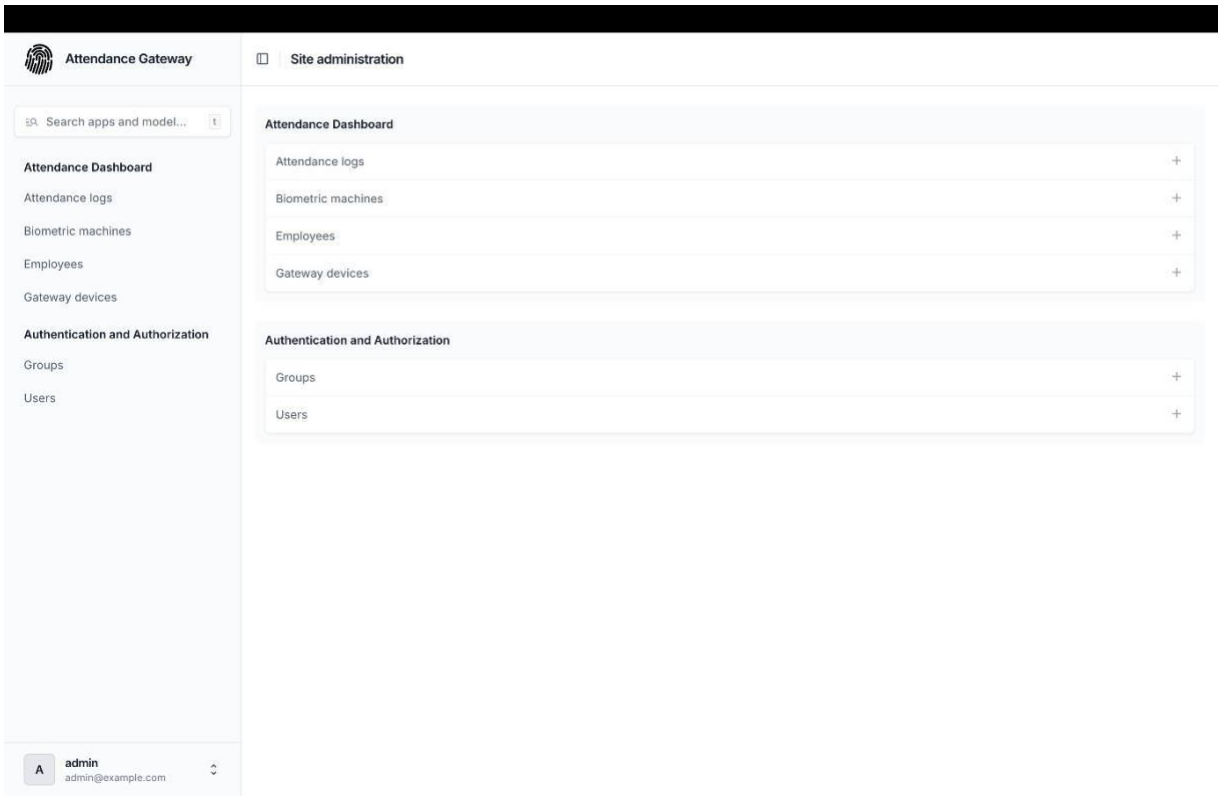


Figure 8.2: System Administration Dashboard Overview

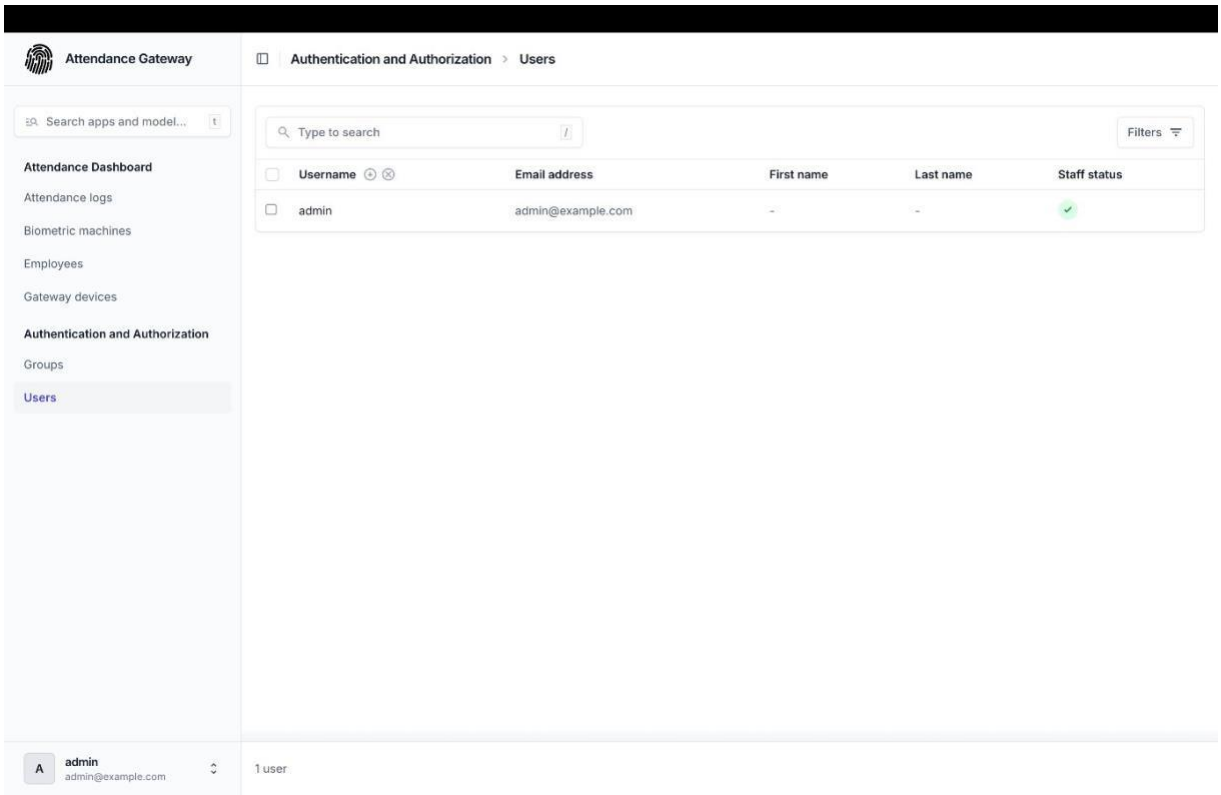


Figure 8.3: User Authentication and Authorization Management Interface

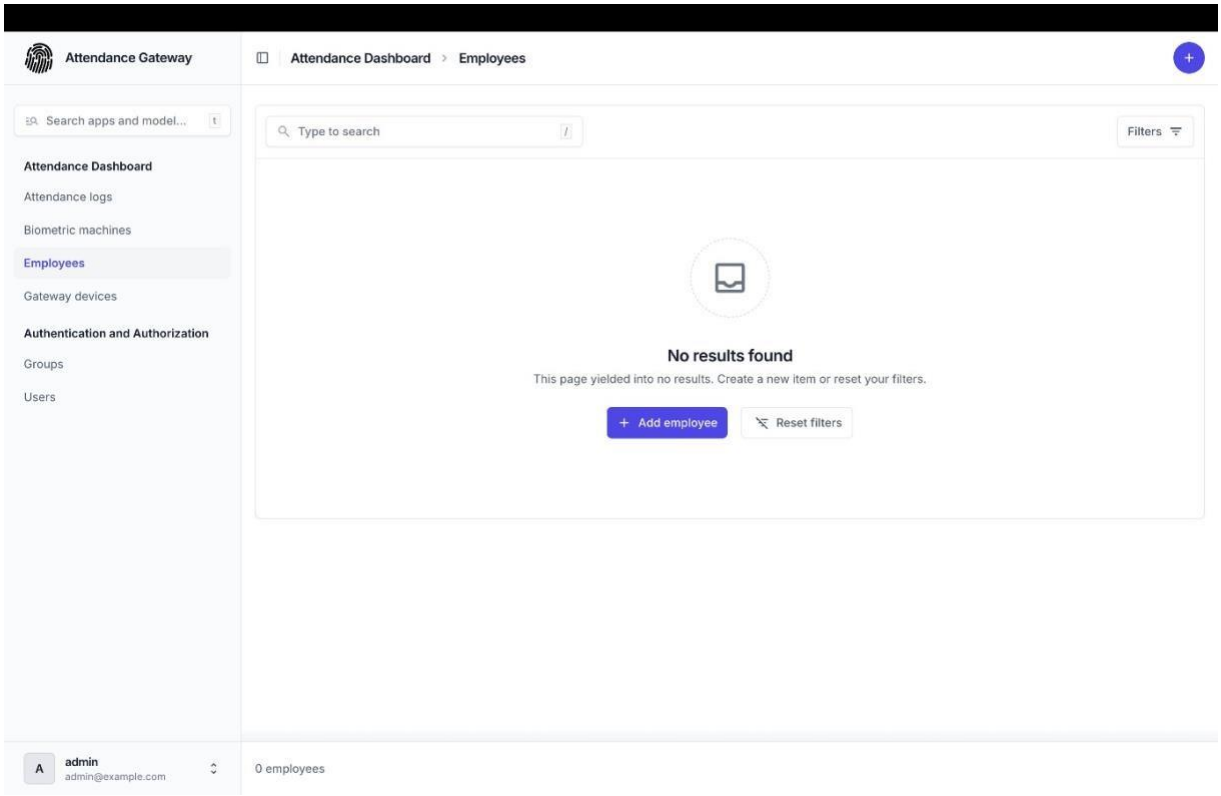


Figure 8.4: Employee Management Module Interface

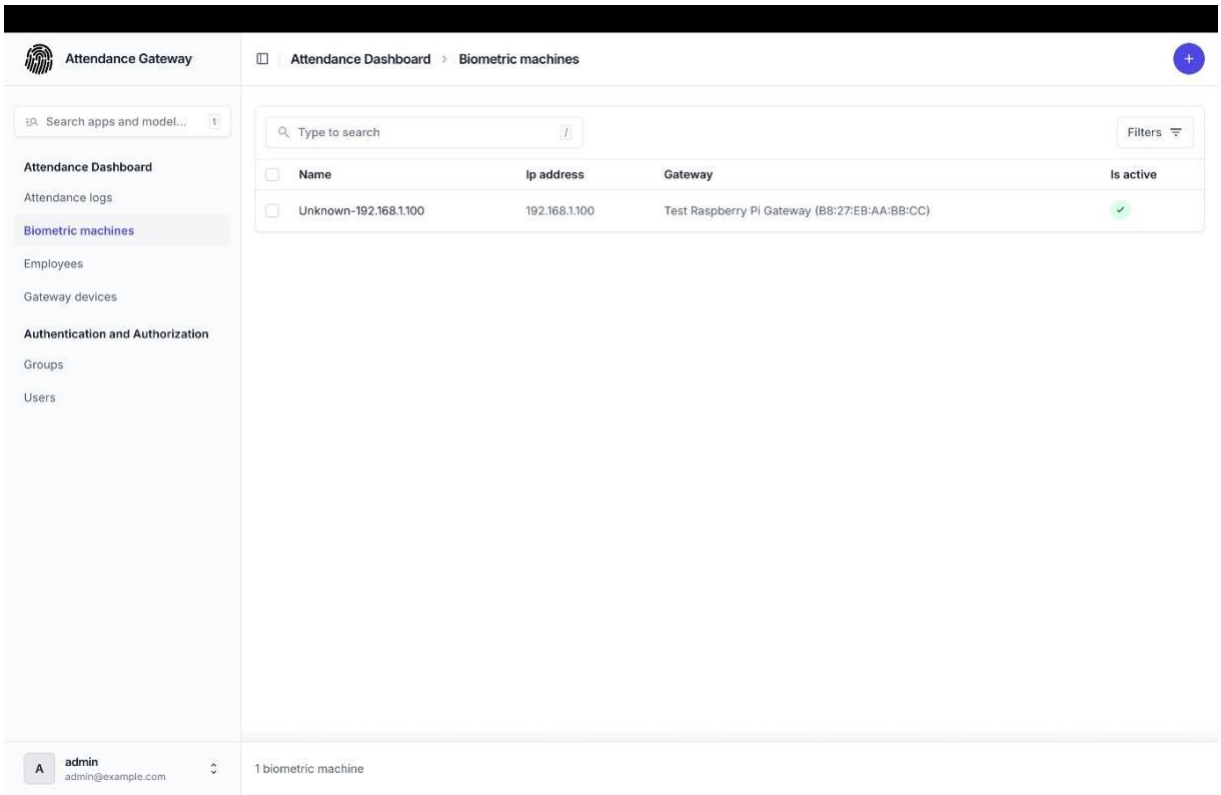


Figure 8.5: Biometric Machine Configuration and Gateway Mapping Interface

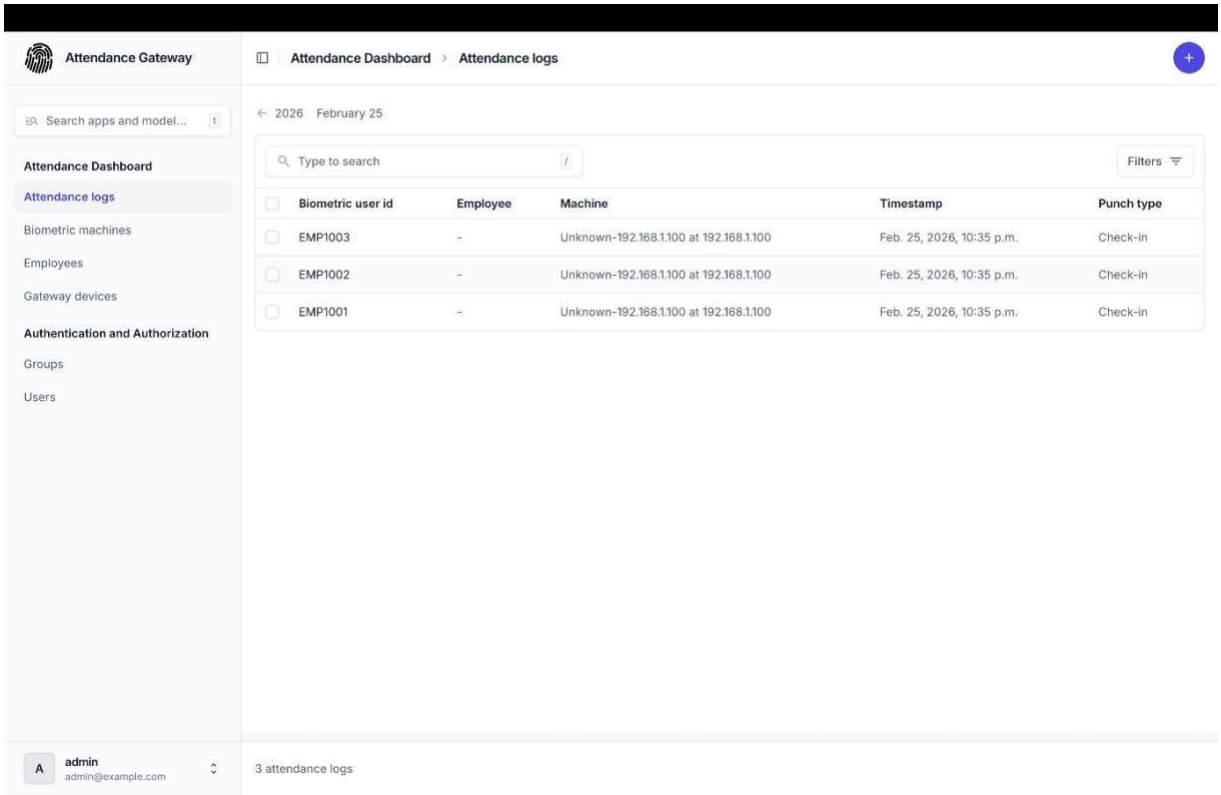
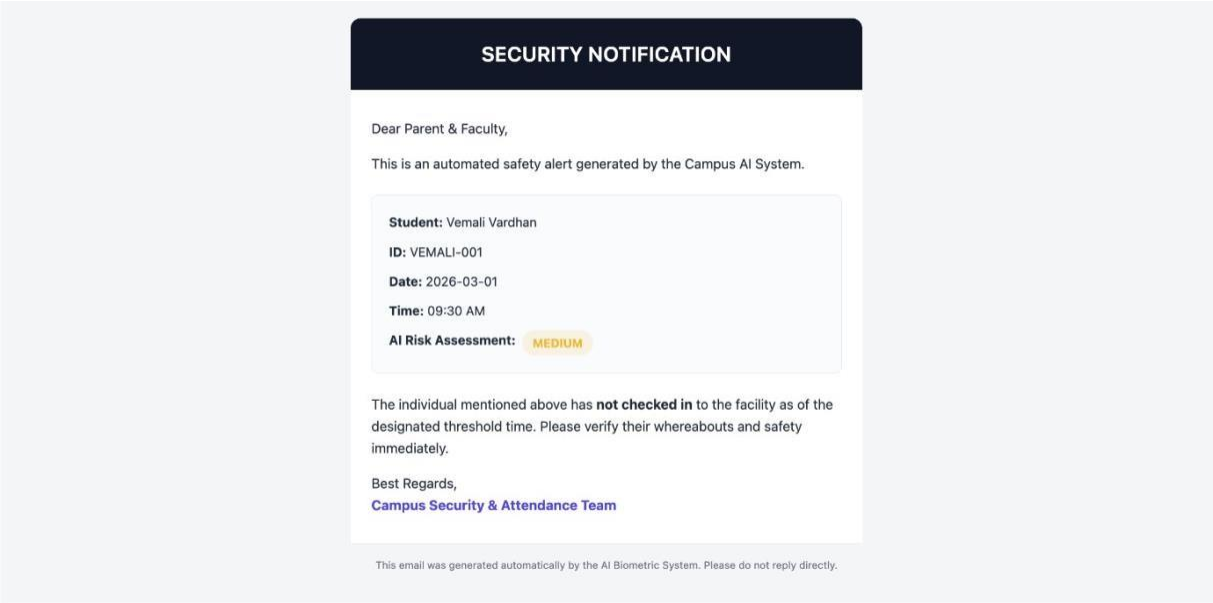


Figure 8.6: Attendance Log Monitoring and Record Verification Interface



SECURITY NOTIFICATION

Dear Parent & Faculty,

This is an automated safety alert generated by the Campus AI System.

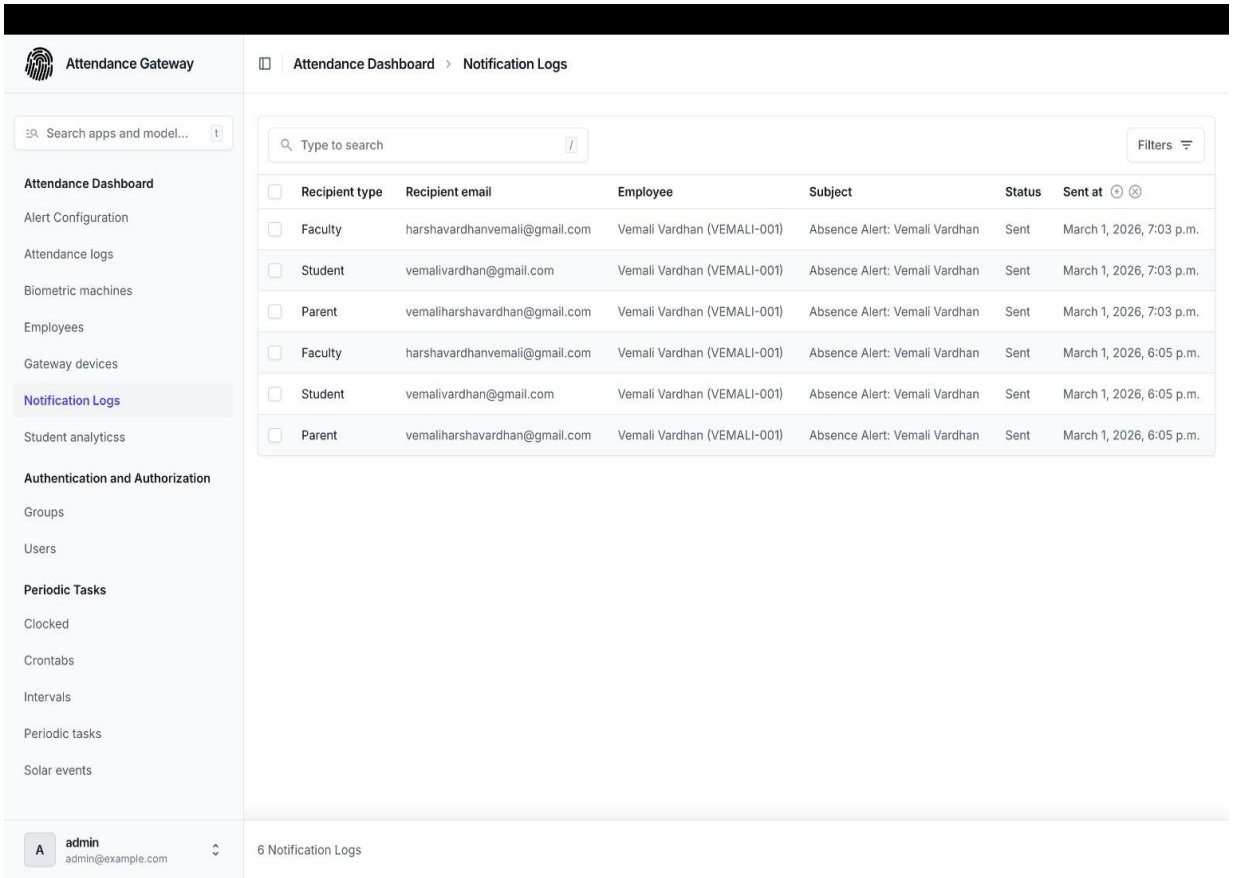
Student: Vemali Vardhan
ID: VEMALI-001
Date: 2026-03-01
Time: 09:30 AM
AI Risk Assessment: MEDIUM

The individual mentioned above has **not checked in** to the facility as of the designated threshold time. Please verify their whereabouts and safety immediately.

Best Regards,
Campus Security & Attendance Team

This email was generated automatically by the AI Biometric System. Please do not reply directly.

Figure 8.7: AI-Generated Security Notification with Risk Assessment Score



Attendance Gateway

Attendance Dashboard > Notification Logs

Search apps and model...

Attendance Dashboard
Alert Configuration
Attendance logs
Biometric machines
Employees
Gateway devices
Notification Logs
Student analyticss
Authentication and Authorization
Groups
Users
Periodic Tasks
Clocked
Crontabs
Intervals
Periodic tasks
Solar events

admin
admin@example.com

Type to search

Filters

Recipient type	Recipient email	Employee	Subject	Status	Sent at
Faculty	harshavardhanvemali@gmail.com	Vemali Vardhan (VEMALI-001)	Absence Alert: Vemali Vardhan	Sent	March 1, 2026, 7:03 p.m.
Student	vemalivardhan@gmail.com	Vemali Vardhan (VEMALI-001)	Absence Alert: Vemali Vardhan	Sent	March 1, 2026, 7:03 p.m.
Parent	vemaliharshavardhan@gmail.com	Vemali Vardhan (VEMALI-001)	Absence Alert: Vemali Vardhan	Sent	March 1, 2026, 7:03 p.m.
Faculty	harshavardhanvemali@gmail.com	Vemali Vardhan (VEMALI-001)	Absence Alert: Vemali Vardhan	Sent	March 1, 2026, 6:05 p.m.
Student	vemalivardhan@gmail.com	Vemali Vardhan (VEMALI-001)	Absence Alert: Vemali Vardhan	Sent	March 1, 2026, 6:05 p.m.
Parent	vemaliharshavardhan@gmail.com	Vemali Vardhan (VEMALI-001)	Absence Alert: Vemali Vardhan	Sent	March 1, 2026, 6:05 p.m.

6 Notification Logs

Figure 8.8: Notification Logs Dashboard

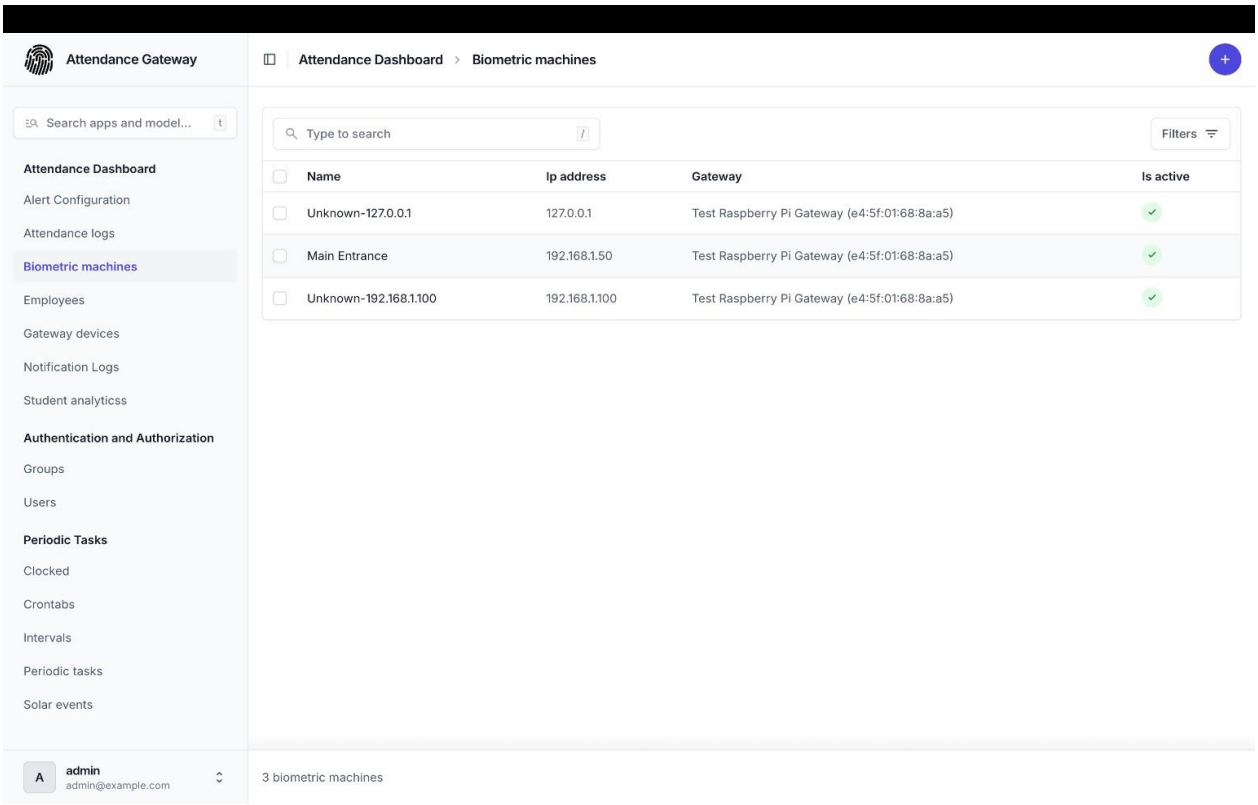


Figure 8.9: Biometric Machines Dashboard

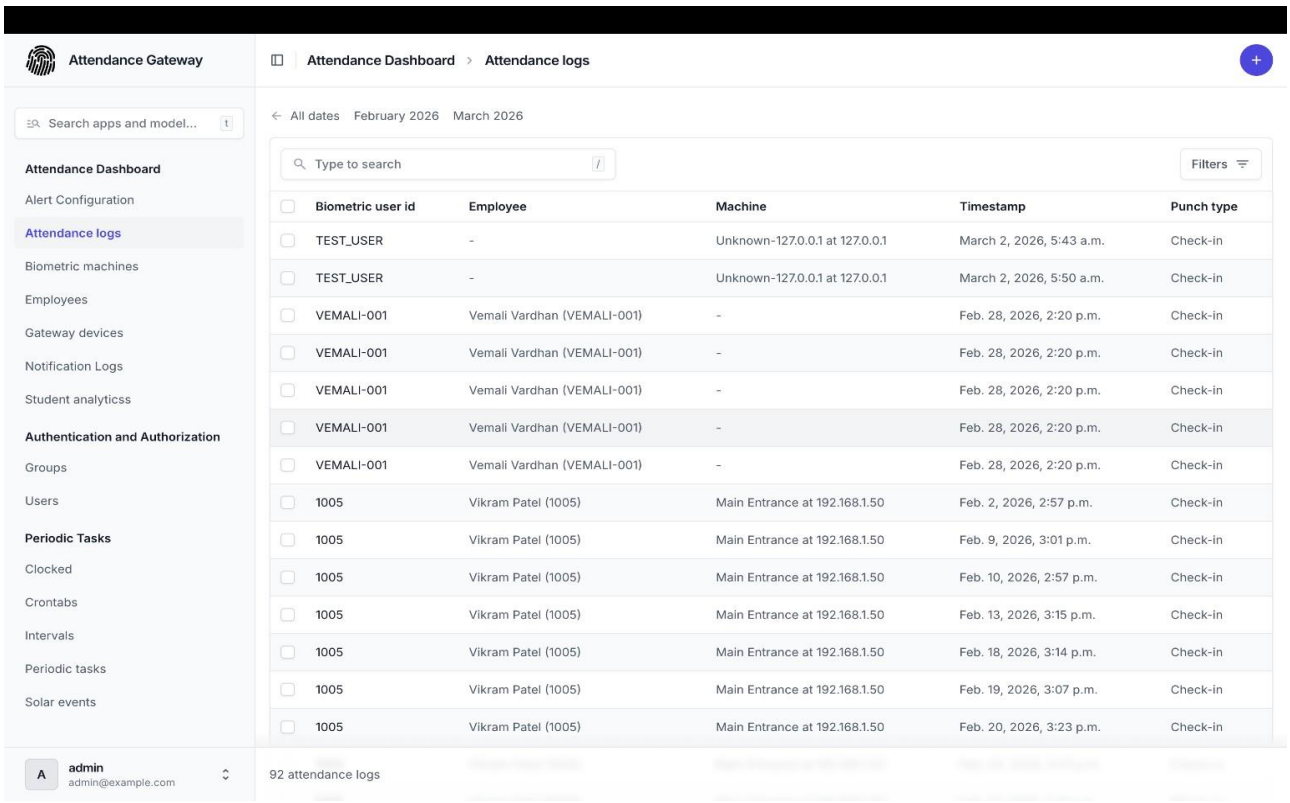


Figure 8.10: Attendance Log Dashboard

8.2 Conclusion and Future Work

This project successfully identified and demonstrated critical security vulnerabilities in existing biometric attendance systems, particularly those related to unencrypted communication and lack of authentication mechanisms. By redesigning the architecture and introducing a Local Secure Gateway, the system now ensures secure validation, encrypted transmission, and controlled synchronization of attendance data.

The proposed hybrid model follows a Zero Trust approach, reducing dependency on insecure local networks and minimizing the risk of unauthorized data manipulation. The system also ensures operational continuity during network outages through local data buffering. Overall, the project enhances the confidentiality, integrity, and availability of sensitive biometric attendance records.

However, certain functional limitations still exist in the current implementation. At present, biometric devices only push attendance logs to the server and do not support command-based communication from the server side. Additionally, the biometric device interface mainly focuses on attendance recording and does not provide contextual feedback or notifications to users. Another limitation is that advanced monitoring and analytical processes are performed only at the cloud server level, which makes the system dependent on internet connectivity for intelligent analysis. Addressing these aspects can further enhance the system's flexibility, responsiveness, and overall functionality. Although the current implementation significantly improves security, there are opportunities for further enhancement. Future work may include:

1. **Bidirectional Device Control (Two-Way Sync):**

   Introduce a Web Socket-based control layer to enable real-time two-way communication between the server and biometric devices.

2. **Intelligent Edge Feedback:**

   Enable the server to send customized messages to the biometric device display after attendance is marked.

3. **Edge-Level Intelligence Deployment:**

   Deploy lightweight predictive or rule-based logic directly on the Raspberry Pi gateway to analyse attendance patterns at the edge.

REFERENCES

[1] J. Galbally, J. Fierrez, and J. Ortega-Garcia, "On the Resilience of Biometric Authentication Systems against Random Inputs," IEEE Transactions on Information Forensics and Security, vol. 7, no. 3, pp. 1124–1136, 2012. doi: 10.1109/TIFS.2012.2199290. URL: https://ieeexplore.ieee.org/document/6197855

[2] A. Alrawais, A. Alhothaily, C. Hu, and X. Cheng, "A Secure, Hybrid, Cloud-Enabled Architecture for Internet of Things," IEEE Access, vol. 5, pp. 258–273, 2017. doi: 10.1109/ACCESS.2017.2654629. URL: https://ieeexplore.ieee.org/document/7823771

[3] National Institute of Standards and Technology (NIST), "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," NIST Special Publication 800-38A, 2001. URL: https://csrc.nist.gov/publications/detail/sp/800-38a/final

[4] M. Abomhara and G. M. Køien, "Security and Privacy in the Internet of Things: Current Status and Open Issues," in Proceedings of the IEEE International Conference on Privacy and Security in Mobile Systems (PRISMS), 2014. doi: 10.1109/PRISMS.2014.6970594. URL: https://ieeexplore.ieee.org/document/6970594

[5] S. Freeman, S. L. Eddy, M. McDonough, M. K. Smith, N. Okoroafor, H. Jordt, and M. P. Wenderoth, "Active Learning Increases Student Performance in Science, Engineering, and Mathematics," Proceedings of the National Academy of Sciences (PNAS), vol. 111, no. 23, pp. 8410–8415, 2014. doi: 10.1073/pnas.1319030111. URL: https://www.pnas.org/doi/10.1073/pnas.1319030111

[6] OWASP Foundation, "OWASP Top 10: Web Application Security Risks," 2021. URL: https://owasp.org/www-project-top-ten/

Appendix: A- Packages/Libraries, Tools used & Working Process

.0

I. Packages / Libraries Used

Python Libraries (Gateway & Backend):

- **PyCryptodome** – Used to implement AES-256 encryption and decryption.
- **Requests** – Used for secure HTTP communication between gateway and cloud server.
- **JSON** – Used for structured data formatting and transmission.
- **Base64** – Used to encode encrypted payload for safe transmission.
- **Datetime / Time** – Used for timestamp generation and replay attack prevention.
- **Threading** – Used for background task execution.

Django & REST Framework:

- **Django** – Backend web framework for developing secure APIs.
- **Django REST Framework (DRF)** – Used to build and validate RESTful APIs.
- **Django ORM** – Used for secure database operations and integrity constraints.

Database:

- **PostgreSQL / SQLite** – Used for storing gateway devices, employees, and attendance logs securely.

II. Tools Used:

- **Raspberry Pi** – Used as a Secure Local Gateway device.
- **eSSL Biometric Machine** – Used for attendance data collection.
- **Wireshark** – Used for packet capture and vulnerability analysis.
- **Notepad++ / VS Code** – Used for coding and debugging.
- **Postman** – Used for API testing and validation.
- **GitHub** – Used for version control and project management.

Appendix: B

Sample Source Code with Execution

SecureSyncModule

```
class SecureSyncClient:
    def _init_(self, server_url, gateway_mac, gateway_api_key):
        self.server_url = server_url
        self.gateway_mac = gateway_mac

        # Ensure the API key correctly acts as a 32-byte AES key string format
        self.aes_key = gateway_api_key.replace('-', '').encode('utf-8')[:32]

    def encrypt_payload(self, records):
        try:
            # Prepare payload
            payload_dict = {
                "logs": records
            }
            json_str = json.dumps(payload_dict)
            json_bytes = json_str.encode('utf-8')
            padded_data = pad(json_bytes, AES.block_size)

            # Generate random Initialization Vector
            iv = os.urandom(16)
            cipher = AES.new(self.aes_key, AES.MODE_CBC, iv)
            encrypted_data = cipher.encrypt(padded_data)

            # Encode for transmission
            iv_b64 = base64.b64encode(iv).decode('utf-8')
            encrypted_b64 = base64.b64encode(encrypted_data).decode('utf-8')

            # Add anti-replay timestamp
            req_timestamp = time.time()
```

```
        return {
            "mac_address":  self.gateway_mac,
            "encrypted_data":  encrypted_b64,
            "iv": iv_b64,
            "timestamp": req_timestamp
        }
    except Exception as e:
        print(f"Encryption error: {e}")
        return None

def sync_to_cloud(self, records):
    if not records:
        return False, 0
    encrypted_payload = self.encrypt_payload(records)
    if not encrypted_payload:
        return False, 0
    try:
        headers = {'Content-Type': 'application/json'}
        response = requests.post(
            self.server_url,
            json=encrypted_payload,
            headers=headers,
            timeout=10
        )
        if response.status_code == 200:
            resp_data = response.json()
            print(f"Sync successful: {resp_data}")
            return True, resp_data.get('logs_processed', 0)
        else:
            print(f"Server rejected sync [{response.status_code}]: {response.text}")
            return False, 0

    except requests.exceptions.RequestException as e:
        print(f"Network error during sync: {e}")
```

```
        return False, 0
```

SecureSyncEndpoint

```
def post(self, request, *args, **kwargs):
    serializer = EncryptedPayloadSerializer(data=request.data)
    if not serializer.is_valid():
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    mac_address = serializer.validated_data['mac_address']
    encrypted_data_b64 = serializer.validated_data['encrypted_data']
    iv_b64 = serializer.validated_data['iv']
    req_timestamp = serializer.validated_data['timestamp']

    # 1. Anti-Replay Check
    current_time = time.time()
    if abs(current_time - req_timestamp) > self.MAX_TIMESTAMP_DIFF:
        return Response({'error': 'Request timestamp is outside the allowed window (Replay
mitigation)'}, status=status.HTTP_403_FORBIDDEN)

    # 2. Retrieve Authenticated Gateway Device
    try:
        device = GatewayDevice.objects.get(mac_address=mac_address, is_active=True)
        device.last_seen = timezone.now()
        device.save(update_fields=['last_seen'])
    except GatewayDevice.DoesNotExist:
        return Response({'error': 'Unauthorized or inactive gateway device.'},
status=status.HTTP_401_UNAUTHORIZED)

    # 3. Decrypt the Payload
    aes_key = str(device.api_key).replace('-', '').encode('utf-8')[:32]

    try:
        iv = base64.b64decode(iv_b64)
        encrypted_data = base64.b64decode(encrypted_data_b64)
```

```
            cipher = AES.new(aes_key, AES.MODE_CBC, iv)
            decrypted_padded = cipher.decrypt(encrypted_data)
            decrypted_json_bytes = unpad(decrypted_padded, AES.block_size)
            decrypted_json_str = decrypted_json_bytes.decode('utf-8')

            payload_data = json.loads(decrypted_json_str)
        except Exception as e:
            return Response({'error': 'Decryption failed.'},
status=status.HTTP_400_BAD_REQUEST)

        # 4. Process the Attendance Logs
        logs_processed = 0
        logs_ignored_duplicates = 0
        new_records = []

        for log_entry in payload_data.get('logs', []):
            try:
                emp_id = log_entry['user_id']
                log_time_str = log_entry['timestamp']
                machine_ip = log_entry['machine_ip']
                machine = BiometricMachine.objects.filter(gateway=device,
ip_address=machine_ip).first()
                if not machine:
                    machine = BiometricMachine.objects.create(name=f"Unknown-{machine_ip}",
gateway=device, ip_address=machine_ip)

                log_datetime = datetime.fromisoformat(log_time_str)
                if timezone.is_naive(log_datetime):
                    log_datetime = timezone.make_aware(log_datetime)
                employee = Employee.objects.filter(biometric_id=emp_id).first()
                _, created = AttendanceLog.objects.get_or_create(
                    biometric_user_id=emp_id,
                    machine=machine,
```

```
                timestamp=log_datetime,
                defaults={
                    'employee': employee,
                    'punch_type': log_entry.get('punch_type', 'Unknown'),
                    'verification_mode': log_entry.get('verification', 0)
                }
            )
            if created:
                logs_processed += 1
                new_records.append({
                    "emp_id": emp_id,
                    "timestamp": log_time_str,
                    "type": log_entry.get('punch_type', 'Unknown')
                })
            else:
                logs_ignored_duplicates += 1
        except Exception as e:
            print(f"Error processing log: {e}")
            pass
```

PayloadValidator

```
class EncryptedPayloadSerializer(serializers.Serializer):
    mac_address = serializers.CharField(max_length=17, required=True, help_text="MAC address of the Gateway Device")
    encrypted_data = serializers.CharField(required=True, help_text="AES encrypted attendance data payload")
    iv = serializers.CharField(required=True, help_text="Initialization Vector for AES decryption base64 encoded")
    timestamp = serializers.FloatField(required=True, help_text="Unix timestamp of the request to prevent replay attacks")
```

DeviceManagementModel

```
class GatewayDevice(models.Model):
    name = models.CharField(max_length=100)
    mac_address = models.CharField(max_length=17, unique=True, help_text="MAC
Address of the Raspberry Pi")
    api_key = models.UUIDField(default=uuid.uuid4, editable=False, unique=True)
    is_active = models.BooleanField(default=True)
    last_seen = models.DateTimeField(auto_now=True)
    created_at = models.DateTimeField(auto_now_add=True)
    def _str_(self):
        return f"{self.name} ({self.mac_address})"

class BiometricMachine(models.Model):
    name = models.CharField(max_length=100)
    ip_address = models.GenericIPAddressField(help_text="Local IP address of the machine")
    port = models.IntegerField(default=4370)
    gateway = models.ForeignKey(GatewayDevice, on_delete=models.CASCADE,
related_name='machines')
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    def _str_(self):
        return f"{self.name} at {self.ip_address}"
```

AttendanceManagementModel

```
class Employee(models.Model):
    biometric_id = models.CharField(max_length=50, unique=True, help_text="ID used in the
biometric machine")
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    email = models.EmailField(blank=True, null=True)
    department = models.CharField(max_length=100, blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return f"{self.first_name} {self.last_name} ({self.biometric_id})"
```

```
class AttendanceLog(models.Model):
    employee = models.ForeignKey(Employee, on_delete=models.CASCADE,
related_name='attendance_logs', null=True, blank=True)
    biometric_user_id = models.CharField(max_length=50, help_text="Raw user ID from the
machine")
    machine = models.ForeignKey(BiometricMachine, on_delete=models.SET_NULL,
null=True, related_name='logs')
    timestamp = models.DateTimeField(help_text="Exact time of the punch")
    punch_type = models.CharField(max_length=20, default="Unknown", help_text="Check-
in, Check-out, etc.")
    verification_mode = models.IntegerField(default=0, help_text="E.g., 0=Password,
1=Fingerprint, 15=Face")
    is_synced = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    class Meta:
        unique_together = ('biometric_user_id', 'machine', 'timestamp')

    def __str__(self):
        return f"{self.biometric_user_id} at {self.timestamp}"
```

PerformanceAnalyticsEndpoint

```
def calculate_performance_score(employee):
    """
    Calculates a Success Score (0-100) based on:
    1. Punctuality (Weight: 40%): logs before 09:05 AM.
    2. Consistency (Weight: 60%): attendance frequency in last 30 days.
    """
    now = timezone.now()
    thirty_days_ago = now - timedelta(days=30)

    logs = AttendanceLog.objects.filter(
        employee=employee,
        timestamp_gte=thirty_days_ago,
        punch_type="Check-in"
```

```
)
total_punches = logs.count()
if total_punches == 0:
    score = 50 # Baseline for new/inactive
    punctuality = 100
    consistency = 0
else:
    # 1. Punctuality Calculation
    on_time_count = 0
    for log in logs:
        # Check if punch was before 09:05 AM (local time)
        # We assume the server is set to the correct timezone
        log_time = log.timestamp.time()
        if log_time.hour < 9 or (log_time.hour == 9 and log_time.minute <= 5):
            on_time_count += 1

    punctuality = (on_time_count / total_punches) * 100

    # 2. Consistency Calculation (Assume 22 working days in 30 days)
    consistency = (total_punches / 22) * 100
    if consistency > 100:
        consistency = 100

    # Weighted Score
    score = (punctuality * 0.4) + (consistency * 0.6)

# 3. Determine Risk Level (Base calculation, potentially overridden by Ollama)
risk = 'LOW'
if score < 40:
    risk = 'HIGH'
elif score < 70:
    risk = 'MEDIUM'

advisory = ""
```

AIAdvisoryEndpoint

```
    response = requests.post(ollama_url, json={
            "model": "llama3.2:1b",
            "prompt": prompt,
            "stream": False,
            "format": "json"
        }, timeout=15)

        if response.status_code == 200:
            raw_response = response.json().get("response", "{}").strip()
            # Strip potential markdown code blocks if the model hallucinates them
            if raw_response.startswith("```json"):
                raw_response = raw_response[7:]
            if raw_response.startswith("```"):
                raw_response = raw_response[3:]
            if raw_response.endswith("```"):
                raw_response = raw_response[:-3]

            ai_data = json.loads(raw_response.strip())
            if "risk_level" in ai_data and ai_data["risk_level"] in ['LOW', 'MEDIUM', 'HIGH']:
                risk = ai_data["risk_level"]
            if "advisory" in ai_data:
                advisory = ai_data["advisory"].upper()
    except Exception as e:
        # Graceful fallback if Ollama is offline or times out
        print(f"[Ollama Error]: {e}")
        advisory = "[SYSTEM FALLBACK] AI ENGINE OFFLINE. DEFAULTING TO
STATISTICAL HEURISTICS."

    # Update or Create Analytics Record
    analytics, _ = StudentAnalytics.objects.update_or_create(
        employee=employee,
        defaults={
            'success_score': int(score),
```

```
        'punctuality_rate': round(punctuality, 1),
        'attendance_consistency': round(consistency, 1),
        'risk_level': risk,
        # We can store advisory later if we add a field, but for now we'll
        # calculate it on the fly or pass it in context if we just need it globally.
        # Currently storing the basic fields to match models.py
    }
)

# Attach advisory to the returned object dynamically so it can be used in views
analytics.ai_advisory = advisory

return analytics
```